

SOFTWARE REQUIRE- MENTS SPECIFICATION (SRS)

AuraMCP

An Intelligent Personal Assistant Platform Integrating Hybrid
Context, Gmail, Telegram, Google Calendar & Web Intelligence

Course Title: Software Engineering and Information System Design Lab

Course Code: CSE-3104

Year & Semester: 3rd Year, 1st Semester

Team Name: Team Axis (Team No: 06)

Document Version: 1.0 — **Date:** 14 June 2026

Submitted To

Md. Samsuddoha

Assistant Professor, Dept. of
CSE, University of Barishal

Prepared By

Name	Student Roll
S.M. Shahriar Surov	230102007
Mahfuj Abdullah	230102019
MD. Jubayer Hasan	230102037
MD. Shihab	230102039
MD. Azad	230102046
MD. Sumon	230102050

TABLE OF CONTENTS

Section Title		Page
0.	Team Contribution	2
1.	Problem Statement	3
2.	Inception	8
3.	Elicitation	15
4.	Use Case Modeling	25
5.	Swimlane Diagrams	31
6.	Structural Modelling	34
7.	Class Design	37
8.	Data Flow Diagrams	38
9.	Behavioral Modelling	40
10.	Non-Functional Requirements	46
11.	Constraints & Assumptions	52
12.	Conclusion	56

Chapter 1

Team Contribution

Name	Key Responsibilities
S.M. Shahriar Sourov	System stability, version control management (Git/GitHub), operational QA, final integration testing, and deployment pipeline.
Mahfuj Abdullah	System architecture design, MCP integration layer, Handler API server skeleton, development workflow definition, and tech-stack selection.
MD. Jubayer Hasan	Backend API development, Prisma ORM implementation, SQL schema optimisation, Gmail and Google Calendar API integration modules.
MD. Shihab	Frontend interface (CLI / web chat UI), state management, project reporting, and user-facing documentation.
MD. Azad	Requirement elicitation, database schema design, SRS authoring, and UML diagram coordination.
MD. Sumon	Unit and integration testing, API validation scripts, bug tracking, news scraper module, and Telegram automation testing.

Chapter 2

Problem Statement

2.1 Introduction to the Problem Domain

The contemporary knowledge worker — student, professional, or researcher — operates across a fragmented constellation of digital tools. A typical day involves checking Gmail for pending replies, opening Google Calendar to confirm meeting times, sending status updates over Telegram, and manually browsing news websites or scrolling through Facebook pages to stay informed. Each of these activities demands a context switch: opening a new application, recalling relevant background information, and composing an appropriate response or action from scratch.

This fragmentation is not merely inconvenient — it has measurable cognitive and economic costs. Research in cognitive psychology consistently shows that task-switching carries a *switching cost*: attention residue from the previous task degrades performance on the current one. For digital communication workers, these costs accumulate across dozens of context switches per day.

The fundamental problem is the absence of a single intelligent agent that can operate across all these channels simultaneously, maintain persistent knowledge about the people involved in each interaction, and act on the user’s behalf with minimal friction.

2.2 Detailed Problem Analysis

2.2.1 Problem 1: No Unified Context Memory

When a user composes an email to a colleague named “Rahim,” they must mentally recall Rahim’s email address, their recent conversation history, Rahim’s role and relationship, and any pending items related to him. This information exists across Gmail threads, calendar entries, and personal memory — but no tool aggregates and presents it automatically before the user begins typing.

Impact: Emails lack context, are mis-addressed, omit important follow-up items, or require the user to spend time searching before composing. Important relation-

ship nuances are lost between interactions.

2.2.2 Problem 2: Email Management is Manual and Reactive

Gmail provides search and labels, but the user must still:

- Manually open and read each thread to assess priority.
- Compose replies from scratch, recalling tone and prior context.
- Track which emails need follow-up without any intelligent reminders.
- Search across hundreds of threads to find a specific piece of information mentioned weeks ago.

There is no proactive intelligence that reads the inbox, identifies what needs attention, drafts appropriate responses, and presents them for one-click approval.

2.2.3 Problem 3: Telegram Communications are Unstructured

Telegram is used for both personal communication and broadcast channels, but all automation is manual. Users who want to send a daily news digest to a group, broadcast a meeting reminder, or trigger a workflow from a command must either do it manually or build custom bots with significant programming effort. There is no accessible, conversational interface to define and manage these automations.

2.2.4 Problem 4: Calendar Management Requires Multiple Steps

Scheduling a meeting with a known contact involves: opening Google Calendar, creating an event, manually typing the contact's email address, enabling Google Meet, setting a reminder, and sending the invite — all as separate manual steps across multiple UI interactions. When the meeting involves multiple contacts whose emails are stored elsewhere, the friction compounds.

2.2.5 Problem 5: News and Social Feed Awareness is Passive

A user interested in staying informed about specific topics, news channels, or Facebook pages must actively visit each source. There is no system that proactively monitors configured sources, filters content by relevance, and delivers a summarised digest through the user's preferred channel (e.g., Telegram) at a scheduled time.

2.3 Problem Domain Diagram

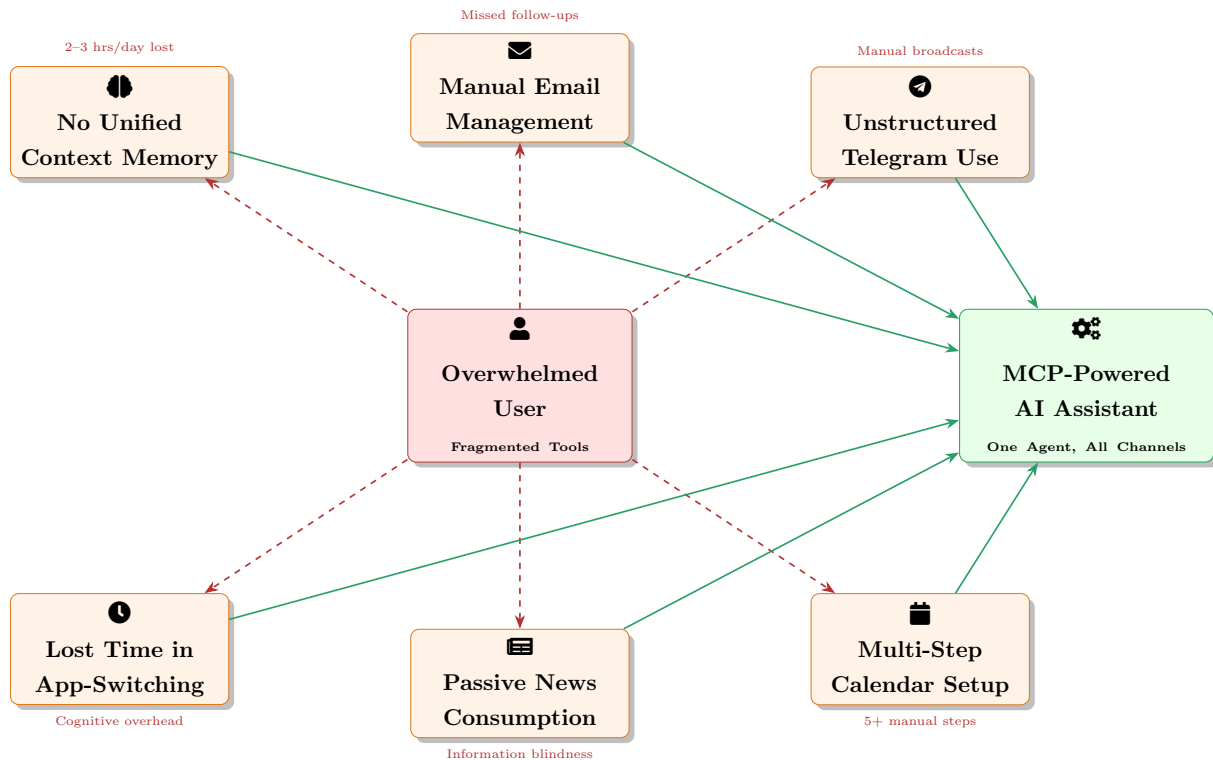


Figure 2.1: Problem Domain Map — Pain Points and Proposed Resolution

2.4 Quantified Impact Analysis

Impact Category	Estimated Daily Cost	Consequence
Email management	45–90 min	Delayed replies, missed follow-ups, context-less communications
Calendar operations	15–30 min	Scheduling errors, missing attendee emails, forgotten Meet links
Telegram management	10–20 min	Manual digests, no automation, missed group updates
News monitoring	20–40 min	Visiting multiple sites manually, no filtering or summarisation
Context recall	15–30 min	Re-reading old emails to recall who someone is and what was agreed

Impact Category	Estimated Daily Cost	Consequence
Total	1.75–3.5 hrs	Compounded productivity loss every working day

2.5 Root Cause Analysis

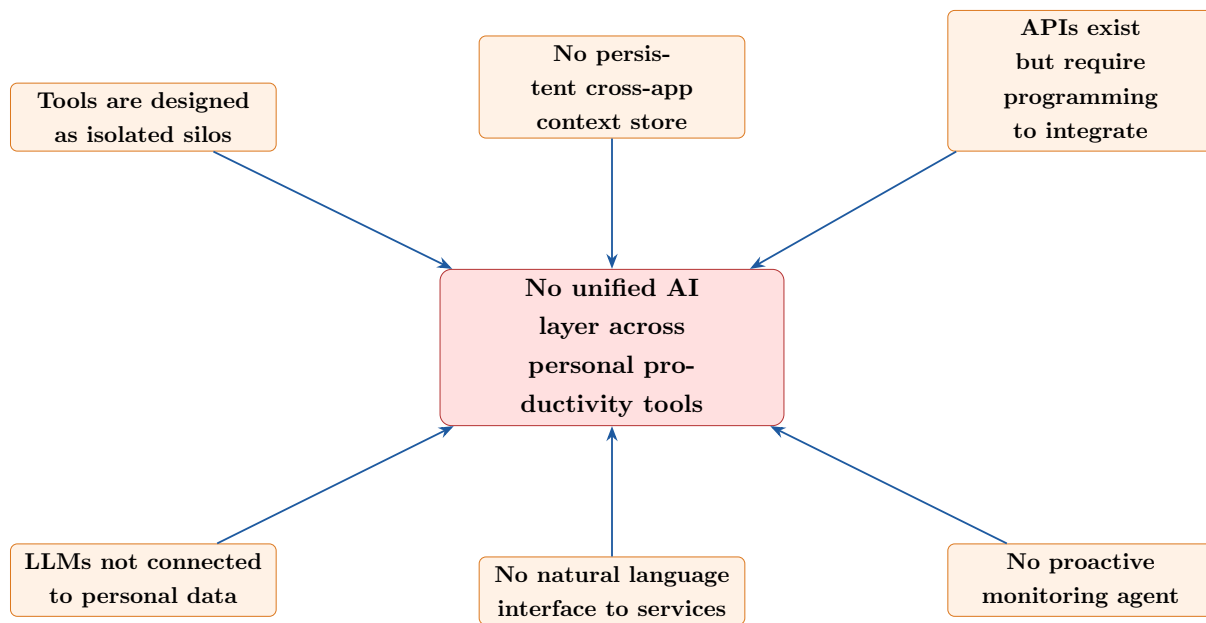


Figure 2.2: Root Cause Analysis — Fishbone (Ishikawa) Summary

2.6 Proposed Solution Statement

Proposed Solution

The **AuraMCP** resolves all identified problems through a three-tier architecture:

1. **MCP Agent (LLM Layer):** A large language model with tool-calling capability serves as the reasoning core. It interprets natural language commands, decides which tools to invoke, sequences multi-step operations, and generates human-readable responses.
2. **MCP Server (Protocol Layer):** An MCP-compliant server maintains a registry of all available tools, routes tool-call requests from the agent to the correct handler, and enforces schema validation on inputs and outputs.
3. **Handler API Server (Integration Layer):** A RESTful API server implements the actual integrations — Gmail API, Google Calendar API, Telegram Bot API,

contact database, and web scrapers — and exposes them as clean, typed endpoints consumed by the MCP Server.

A **Hybrid Interaction Manager (HIM)** runs as a cross-cutting concern, injecting relevant contact profiles and interaction history into the agent’s context before every inference call, ensuring the agent always knows *who* it is dealing with.

Chapter 3

Inception

3.1 Project Vision Statement

Vision

To create a single, conversational AI agent that acts as a personal operations hub — knowing the user’s contacts deeply, monitoring their inbox and calendar proactively, automating their messaging workflows, and keeping them informed about the world — all through natural language, with zero manual app-switching and persistent context across every interaction.

3.2 Project Goals and Objectives

3.2.1 Primary Goals

1. **Eliminate context-switching:** The user shall be able to accomplish all primary communication and scheduling tasks from a single conversational interface without opening Gmail, Google Calendar, or Telegram separately.
2. **Persistent person-awareness:** The system shall maintain a rich contact database and automatically inject relevant person profiles into every interaction, replicating the way a human assistant “knows” who people are.
3. **Proactive intelligence:** The system shall not wait to be asked — it shall push news digests, send reminders, and surface important inbox items according to user-configured schedules.
4. **Protocol-native extensibility:** By building on MCP, new integrations (Slack, WhatsApp, Notion) shall be addable as new tools without modifying the core agent or server.

3.2.2 Secondary Objectives

- Full audit trail of all agent actions for user review.
- Explicit confirmation step before any write operation (send email, create event, delete

record).

- Graceful degradation: failure of one integration must not affect others.
- All credentials stored securely; zero plaintext secrets in logs or version control.

3.3 Technology Stack

Layer	Technology	Justification
MCP Agent	Claude 3.5 Sonnet / GPT-4o	State-of-the-art tool-use support; large context window (200k tokens) for HIM injection
MCP Server	MCP SDK (Python/TypeScript)	Official Anthropic-maintained SDK; standardised tool schema, transport, and life-cycle
Handler API	FastAPI (Python)	Async-native, auto-generated OpenAPI docs, Pydantic validation, excellent Google API client support
Database	SQLite + SQLAlchemy / Prisma	Lightweight, zero-server persistence; sufficient for single-user contact and cache store
Gmail Integration	Google Gmail API v1	Official API; OAuth 2.0; full CRUD on messages and threads
Calendar Integration	Google Calendar API v3	Official API; Meet conference data generation; attendee invites
Telegram	python- telegram-bot / Telegraf	Mature libraries; webhook and polling support; rich command handling
Scraper	BeautifulSoup4 + httpx	Lightweight HTML parsing; async HTTP; sufficient for public news pages and FB pages
Scheduler	APScheduler	In-process cron scheduling; persistent job store; no external dependency
Auth	OAuth 2.0 + google-auth library	Industry standard; supports offline refresh tokens for server-side use

3.4 Feasibility Study

3.4.1 Technical Feasibility

All required APIs are mature, publicly documented, and actively maintained:

- **Gmail API:** 6+ years in production; 1B quota units/day free; Python client library available.
- **Google Calendar API:** Full event CRUD and Meet conference data generation; well-documented OAuth scopes.
- **Telegram Bot API:** Free, no quota limits for standard usage; webhook and long-polling both supported.
- **MCP SDK:** Open-source, actively maintained by Anthropic; Python and TypeScript implementations available.
- **LLM Tool Use:** Claude 3.5 Sonnet and GPT-4o both have production-grade tool-calling with JSON schema validation.

Integration of these components within a 3-tier architecture is technically straightforward. The primary technical risk is the fragility of web scraping for Facebook pages (see Risk Register, Section 2.7).

3.4.2 Operational Feasibility

The system requires:

- A server or local machine running Python 3.11+ with 512 MB RAM.
- Google Cloud Console project with Gmail and Calendar APIs enabled and OAuth 2.0 credentials configured.
- A Telegram Bot created via BotFather with the token stored as an environment variable.
- An LLM API key (Anthropic or OpenAI) with sufficient credit.

Operational maintenance is limited to periodic OAuth token refresh, scraper rule updates when target websites restructure their HTML, and LLM prompt tuning as model versions update.

3.4.3 Economic Feasibility

Cost Item	Estimated Monthly Cost	Notes
Gmail API	\$0	Free within quota
Google Calendar	\$0	Free within quota
Telegram Bot API	\$0	Fully free
LLM API (Claude)	\$2–\$8	Depends on usage; ~1M tokens/month
Hosting (VPS)	\$5–\$10	DigitalOcean Droplet / Railway
Domain (optional)	\$1	For webhook HTTPS endpoint
Total	\$8–\$19/month	Well within student budget

3.5 Stakeholder Analysis

Stakeholder	Role	Primary Interest	Success Criterion
Primary User	Owner and sole operator	Seamless productivity automation	All tasks completable by voice/text with no app-switching
Course Supervisor	Academic evaluator	Correct SE process, SRS quality	Full IEEE 830 compliance, all diagrams present
Google APIs	External service provider	Quota compliance, policy adherence	No rate limit violations, proper OAuth scope usage
Telegram Platform	External service provider	Bot API compliance	No spam, rate limits respected
LLM Provider (Anthropic)	AI inference backend	Correct tool-call format	Valid JSON schemas, token usage within plan limits
Open Source Community	Indirect contributor	Proper attribution	All dependencies listed in requirements.txt

3.6 System Context Diagram

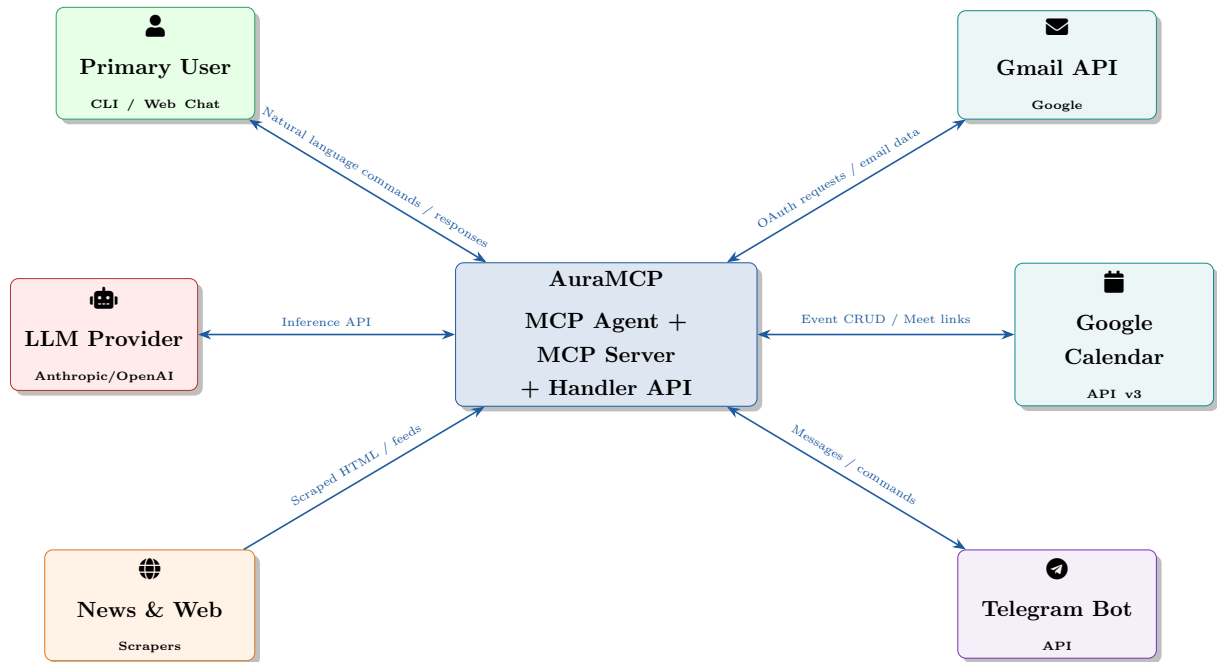


Figure 3.1: System Context Diagram — External Actors and Data Flows

3.7 High-Level Architecture Diagram

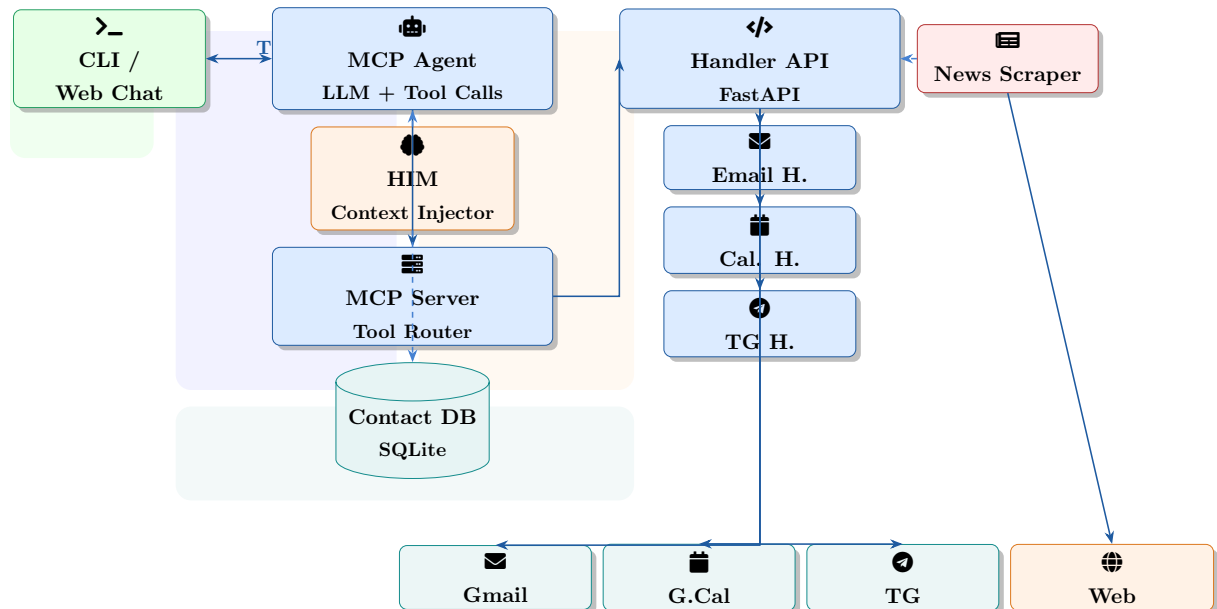


Figure 3.2: High-Level Four-Tier Architecture Diagram

3.8 Risk Register

Risk	Likelihood	Impact	Priority	Mitigation Strategy
OAuth token expiry	Medium	High	P1	Implement automatic re-fresh token rotation; alert user on failure
LLM hallucination on tool calls	Medium	High	P1	Strict Pydantic schema validation; re-prompt on parse failure (max 3 retries)
Gmail quota exceeded	Low	High	P2	Request batching; exponential back-off; cache last N emails locally
Facebook scraping blocked	High	Medium	P2	Pluggable scraper interface; RSS/Atom fallback; user-configurable selectors
Context window overflow	Medium	Medium	P2	HIM summarisation; rolling window; priority-ranked context injection
Telegram Bot rate limit	Low	Low	P3	Message queue with 30 msg/sec limiter; retry with jitter
LLM API downtime	Low	High	P2	Configurable fallback provider (Claude → GPT-4o)
Data loss on server restart	Low	High	P1	SQLite WAL mode; daily backup to local file; startup integrity check
Google Meet link generation failure	Low	Medium	P3	Retry once; if failed, create event without Meet and notify user
News scraper site restructure	Medium	Low	P3	CSS selector stored in DB (not hardcoded); user can update via command

3.9 Project Milestones

Week	Milestone	Deliverables
1–2	Project Setup	Repository, environment config, Google Cloud project, OAuth setup, Telegram bot creation
3–4	HIM Module	Contact CRUD API, context injection engine, SQLite schema, unit tests
5–6	Email Module	Gmail fetch, summarisation, draft/send/reply handlers, email cache, integration tests
7–8	Calendar Module	Event CRUD, Meet link generation, contact-to-email resolution, reminder scheduling
9–10	Telegram Module	Send/receive, command handling, digest scheduling, group broadcast
11	News Scraper	Multi-source scraper, keyword filter, Telegram push integration
12	MCP Integration	Full MCP Server tool registry, agent end-to-end tests, context injection tuning
13	System Testing	Full integration test suite, load testing, edge case handling
14	Documentation	SRS finalisation, API docs, user guide, deployment guide

Chapter 4

Elicitation

4.1 Elicitation Methodology

Requirements were gathered through four complementary techniques:

1. **Scenario-Based Analysis:** The team walked through a full working day of the target user, identifying every communication and scheduling action and asking: “Could the assistant handle this? What would it need to know?”
2. **Competitive Benchmarking:** Existing tools were analysed — Superhuman (email), Notion AI (knowledge management), Zapier (automation), IFTTT, Google Assistant, and ChatGPT with plugins — to identify gaps and features worth adopting.
3. **MCP Architecture Constraint Analysis:** Since the system must be MCP-native, all capabilities were expressed as tool definitions first, then requirements were derived from what each tool needs to do and what context it needs.
4. **Team Brainstorming:** Structured sessions where each team member proposed requirements from personal pain-points, followed by group prioritisation using MoSCoW.

4.2 MoSCoW Prioritisation

ID	Requirement	Priority	Rationale
FR-H1	Store and retrieve contact profiles	Must	Core of HIM; all modules depend on it
FR-H2	Context injection before inference	Must	Defines the system’s key differentiator
FR-H3	Auto-detect person mentions	Must	Required for seamless UX
FR-H4	Update/delete contacts	Must	Basic CRUD completeness
FR-H5	Relationship tagging	Should	Enhances context quality
FR-H6	Interaction history logging	Should	Enables memory across sessions

ID	Requirement	Priority	Rationale
FR-H7	Contact import from Gmail	Could	Convenience feature for setup
FR-E1	Fetch inbox emails	Must	Foundation of email module
FR-E2	Summarise email threads	Must	Core value proposition
FR-E3	Draft emails via NL	Must	Core value proposition
FR-E4	Send emails	Must	Required for module completeness
FR-E5	Reply to threads	Must	High-frequency use case
FR-E6	Context-aware composition	Must	Differentiates from standard email
FR-E7	Search and label emails	Should	Important but not blocking
FR-E8	Rolling inbox awareness cache	Should	Performance optimisation
FR-E9	Attachment handling	Won't	Out of scope for v1.0
FR-C1	Create calendar events	Must	Core calendar feature
FR-C2	Generate Google Meet links	Must	Key scheduling differentiator
FR-C3	Invite contacts by name	Must	Requires HIM integration
FR-C4	Set reminders	Must	Prevents missed meetings
FR-C5	Query upcoming events	Must	Common daily use case
FR-C6	Update and delete events	Should	CRUD completeness
FR-C7	Smart scheduling suggestions	Could	Advanced feature; v2.0 candidate
FR-C8	Recurring events	Won't	Scope risk in v1.0
FR-T1	Send Telegram messages	Must	Core channel output
FR-T2	Receive and process messages	Must	Required for command handling
FR-T3	Scheduled digest summaries	Must	Key automation feature
FR-T4	Slash command automation	Must	Power user feature
FR-T5	Group/channel broadcast	Should	Enables team use cases
FR-T6	Inline news push to Telegram	Should	Connects news and Telegram modules
FR-N1	Scrape configured news websites	Must	Core news feature
FR-N2	Monitor Facebook pages	Must	High-value use case

ID	Requirement	Priority	Rationale
FR-N3	Keyword/topic filtering	Must	Required for relevance
FR-N4	On-demand digest in conversation	Must	Basic interaction mode
FR-N5	Scheduled Telegram news push	Must	Proactive intelligence
FR-N6	Add/remove/pause sources via NL	Should	UX improvement
FR-N7	RSS/Atom feed support	Could	Fallback for FB scraping

4.3 Full Functional Requirements

4.3.1 Module 1 — Hybrid Interaction Manager (HIM)

ID	Name	Full Description
FR-H1	Store Contact Profiles	The system shall maintain a persistent SQLite database of contact records. Each record shall contain: unique ID, full name, list of email addresses, phone number(s), relationship tag (free text), organisation, notes field, and timestamps for creation and last update.
FR-H2	Context Injection	Before every LLM inference call, the HIM module shall query the contact database for all persons mentioned in the current conversation turn and inject their full profile as a structured block into the system prompt. The injected block shall include name, emails, phone, tag, notes, and the last three interaction log entries.
FR-H3	Auto-Detect Person Mentions	The system shall maintain an in-memory name/email index and scan each incoming user message for matches. On detection, the matching contact profile shall be retrieved and marked for context injection without requiring the user to explicitly request it.

ID	Name	Full Description
FR-H4	Add Contacts via Natural Language	The system shall parse natural language contact creation commands (e.g., “Add Karim, email karim@email.com, colleague at BUET”) and create the corresponding database record. The system shall confirm creation and display the stored record.
FR-H5	Update Contacts via Natural Language	The system shall parse update commands (e.g., “Update Karim’s phone to 01712345678”) and apply the change to the existing record. The system shall display before/after values and request confirmation for destructive changes.
FR-H6	Delete Contacts	The system shall support contact deletion with mandatory confirmation prompt. Deletion shall cascade to interaction log entries.
FR-H7	Relationship Tagging	Contacts shall be assignable to relationship tags (colleague, client, family, friend, vendor). Tag-based queries shall be supported (e.g., “Show me all my clients”).
FR-H8	Interaction History Logging	Every agent action involving a named contact (email sent, event created, Telegram message sent) shall be logged as an Interaction-Log record with: contact ID, action type, short summary, and UTC timestamp.
FR-H9	Query Interaction History	The system shall support queries like “What have I done with Karim recently?” and return a formatted summary of the last N interaction log entries for that contact.
FR-H10	Contact Search	The system shall support fuzzy name search and exact email/phone search across the contact database, returning ranked results.

4.3.2 Module 2 — Email Module (Gmail)

ID	Name	Full Description
FR-E1	Fetch Inbox	The system shall fetch the N most recent emails from the Gmail inbox (default N=20, configurable). Each fetched email record shall include: message ID, thread ID, sender name and address, recipient list, subject, date, plain-text body snippet (first 500 chars), and label list.
FR-E2	Fetch Full Thread	Given a thread ID, the system shall fetch all messages in the thread in chronological order, including full body text (plain/text preferred, HTML stripped).
FR-E3	Summarise Single Email	The system shall generate a structured natural language summary of a single email including: who sent it, when, what they want or say, and whether any action is required.
FR-E4	Summarise Thread	The system shall generate a flowing narrative summary of a full email thread, capturing the arc of the conversation, key decisions, outstanding action items, and tone.
FR-E5	Inbox Triage	On request (“What needs my attention?”), the system shall scan the N most recent unread emails, classify each as Urgent / Normal / FYI, and present a prioritised digest.
FR-E6	Draft Email from Natural Language	Given a recipient name or email, a subject intent, and bullet-point content, the system shall compose a well-structured email draft. If the recipient is a known contact, HIM context (relationship, history, tone) shall influence the draft.
FR-E7	Send Email	The system shall send a composed draft via the Gmail API after explicit user confirmation. The system shall log the send action against the recipient’s contact record.

ID	Name	Full Description
FR-E8	Reply to Thread	Given a thread ID and reply intent, the system shall fetch the full thread context, compose a contextually appropriate reply, present it for review, and send it on confirmation. Thread continuity (In-Reply-To headers) shall be preserved.
FR-E9	Forward Email	The system shall support forwarding a message to a new recipient with an optional preamble, after confirmation.
FR-E10	Search Emails	The system shall translate natural language search queries (e.g., “Emails from Karim last month about the report”) into Gmail API query strings and return matching results.
FR-E11	Apply Labels	The system shall support creating, applying, and removing Gmail labels on messages and threads via natural language commands.
FR-E12	Email Cache	The system shall maintain a local cache of the last 50 fetched email metadata records (not full bodies) in SQLite to reduce API calls for repeated queries within the same session.

4.3.3 Module 3 — Google Calendar Module

ID	Name	Full Description
FR-C1	Create Event	The system shall create a Google Calendar event given: title, start datetime, end datetime (or duration), optional description, and optional location. All fields shall be extractable from natural language input.
FR-C2	Generate Meet Link	When creating a meeting-type event, the system shall automatically request Google Meet conference data via the Calendar API and attach the generated Meet link to the event.

ID	Name			Full Description
FR-C3	Invite	Named	Con-	The system shall resolve contact names mentioned in the meeting command to their stored email addresses using HIM, add them as attendees in the Google Calendar event, and trigger invite emails via Google Calendar’s notification system.
FR-C4	Invite by Email			The system shall also support direct email address input for invites without requiring a stored contact record.
FR-C5	Set Reminders			The system shall add popup and/or email reminders to events at user-specified intervals (e.g., “15 minutes before”, “1 day before”).
FR-C6	Query	Upcoming		The system shall return a formatted list of upcoming events for: today, tomorrow, this week, next week, or a specific date range. Each entry shall show title, time, attendees, and Meet link if present.
	Events			
FR-C7	Update Event			The system shall support rescheduling (change time/date), renaming, adding/removing attendees, and updating description of existing events via natural language.
FR-C8	Delete / Cancel Event			The system shall support event cancellation with confirmation. Cancelled events shall trigger cancellation notifications to attendees via Google Calendar.
FR-C9	Check for Conflicts			Before creating a new event, the system shall query the calendar for the requested time slot and warn the user of any existing conflicting events.
FR-C10	Log Calendar Actions			All created, updated, and cancelled events involving named contacts shall be logged in the HIM interaction log.

4.3.4 Module 4 — Telegram Module

ID	Name	Full Description
FR-T1	Send Text Message	The system shall send a plain-text Telegram message to a specified chat ID, username, or stored contact (resolved via HIM) on user command.
FR-T2	Send Formatted Message	The system shall support Telegram MarkdownV2 formatting: bold, italic, code blocks, inline links. The agent shall apply appropriate formatting when composing structured messages.
FR-T3	Receive and Parse Messages	The system shall receive incoming Telegram messages via webhook or long-polling and make them available to the MCP Agent for processing.
FR-T4	Scheduled Digest — Daily	The system shall send an automatically generated daily summary to a configured Telegram chat at a user-defined time (default: 08:00 local). The digest shall cover: today's calendar events, unread email summary, and top news items.
FR-T5	Scheduled Digest — Weekly	The system shall optionally send a weekly summary every Monday covering the prior week's key interactions and next week's calendar.
FR-T6	Slash Command: /summary	On receiving <code>/summary</code> , the system shall generate and send a real-time inbox + calendar summary to the requesting chat.
FR-T7	Slash Command: /news	On receiving <code>/news</code> , the system shall trigger a fresh scrape and deliver the latest news digest.
FR-T8	Slash Command: /remind	On receiving <code>/remind [time] [message]</code> , the system shall create a one-off reminder that sends the message at the specified time.
FR-T9	Slash Command: /contacts	On receiving <code>/contacts [name]</code> , the system shall return the contact profile for the queried name.
FR-T10	Group Broadcast	The system shall support sending a message to a Telegram group or channel where the bot has admin rights.

ID	Name	Full Description
FR-T11	Log Telegram Actions	All outbound messages to named contacts shall be logged in the HIM interaction log.

4.3.5 Module 5 — News & Web Context Module

ID	Name	Full Description
FR-N1	Scrape News Websites	The system shall scrape configured news website URLs at scheduled intervals (default: every 2 hours). For each article, it shall extract: title, publication date, URL, and lead paragraph summary. CSS selectors for each site shall be stored in the NewsSource table.
FR-N2	Monitor Facebook Pages	The system shall scrape the public post feed of configured Facebook page URLs. For each post, it shall extract: post text (up to 500 chars), post date, and post URL.
FR-N3	RSS/Atom Feed Support	As a fallback for sites that provide RSS/Atom feeds, the system shall parse feed XML and extract equivalent fields.
FR-N4	Keyword Filtering	The system shall maintain per-source keyword lists. Only articles/posts containing at least one configured keyword shall be included in digests. An empty keyword list means “include everything.”
FR-N5	Deduplication	The system shall track article URLs and post IDs already delivered and shall not repeat items across digest cycles.
FR-N6	On-Demand Digest	On user request, the system shall trigger an immediate scrape cycle (or use cached results if the last scrape was within 30 minutes) and present a formatted digest in the conversation.
FR-N7	Scheduled Push Telegram	The system shall push news digests to a configured Telegram chat at user-defined cron times. The digest shall be formatted for Telegram MarkdownV2 with article titles as inline links.

ID	Name			Full Description
FR-N8	Manage	Sources	via NL	The system shall support adding a new source (“Add prothomalo.com to my news sources”), removing a source, and pausing/resuming a source via natural language commands.
FR-N9	Summarise Articles			When the user requests more detail on a news item, the system shall fetch the full article text and generate a 3–5 sentence summary via the LLM.

Chapter 5

Use Case Modeling

5.1 Actor Definitions

Actor	Description
Primary User	The single authenticated human who interacts with the system through a CLI, web chat, or Telegram. All direct commands originate from this actor.
MCP Agent	The AI reasoning layer. Acts as a secondary system actor that orchestrates tool calls on behalf of the Primary User.
Scheduler (Cron)	A time-based trigger actor that fires automated tasks (digest generation, news scraping) without user initiation.
Gmail API	External actor representing Google's Gmail service. Provides and receives email data.
Google Calendar API	External actor representing Google's Calendar service. Manages events and Meet conference data.
Telegram Bot API	External actor representing Telegram's bot infrastructure. Routes outbound messages and inbound commands.
News Sources / Facebook	External actor representing the scraped websites and public Facebook pages.

5.2 Master Use Case Diagram

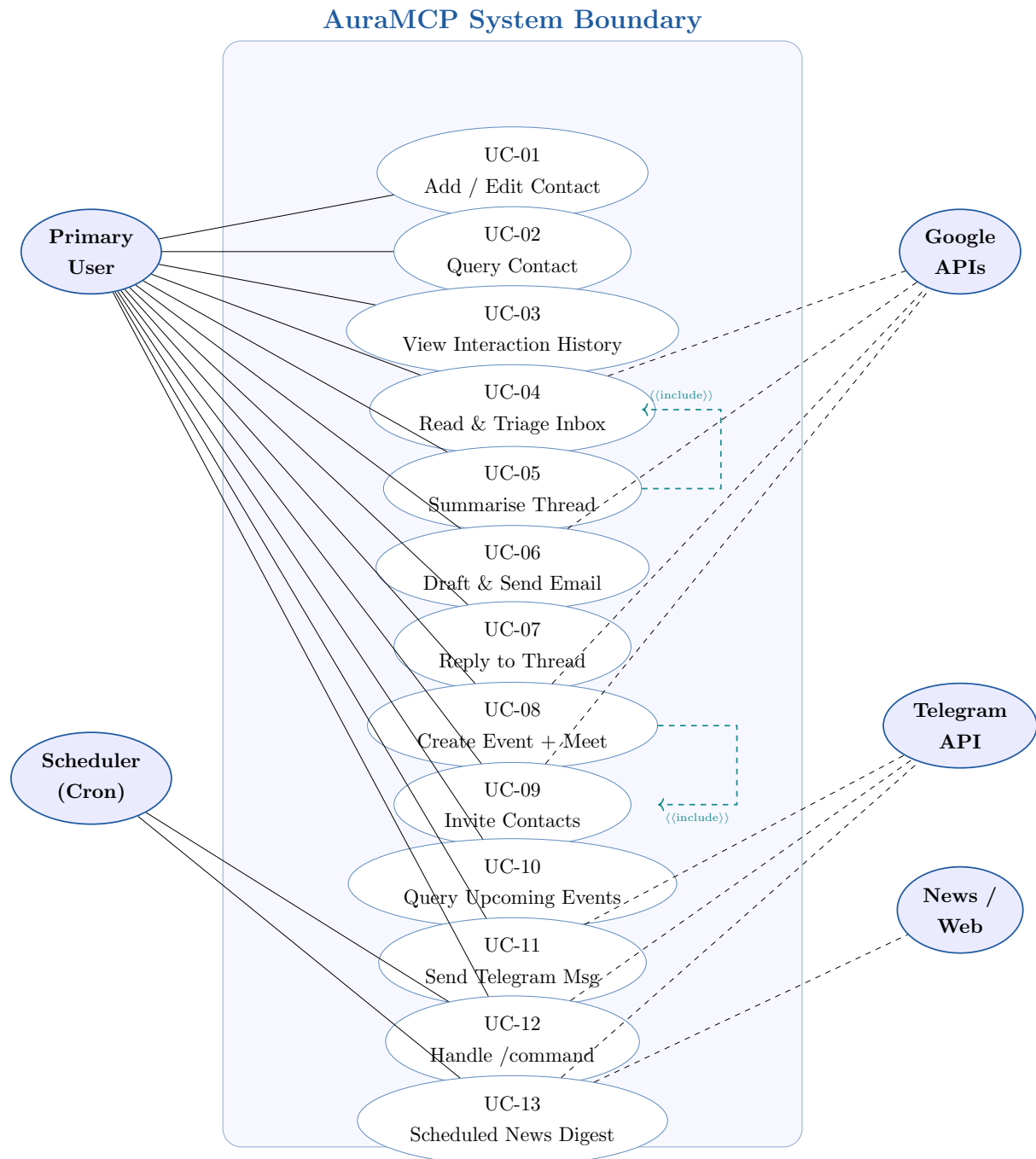


Figure 5.1: Master Use Case Diagram — All Actors and Use Cases

5.3 Detailed Use Case Descriptions

5.3.1 UC-01: Add / Edit Contact

Field	Detail
Use Case ID	UC-01
Name	Add / Edit Contact
Actor(s)	Primary User
Precondition	User is authenticated; HIM database initialised
Trigger	User provides a create or update command in natural language
Main Success Flow	1. User: “Add Rahim, email rahim@gmail.com, tag colleague, works at BUET.” 2. System parses name, email, tag, and notes fields. 3. System checks for duplicate (same name or email). 4. System inserts record into Contact table. 5. System responds: “Contact Rahim saved. Email: rahim@gmail.com.”
Alternate Flow A	3a. Duplicate found → System asks: “Rahim already exists. Update? (yes/no)” 3b. User confirms → System merges new fields into existing record.
Alternate Flow B	2a. Missing required field (name or email) → System asks for the missing field.
Postcondition	Contact record persisted; available for HIM injection.
Business Rule	Every contact must have at least one name field. Email is strongly recommended.

5.3.2 UC-06: Draft & Send Email

Field	Detail
Use Case ID	UC-06
Name	Draft & Send Email
Actor(s)	Primary User, Gmail API
Precondition	Gmail OAuth token valid; HIM populated
Trigger	User requests to send an email
Main Success Flow	1. User: “Email Rahim about the project deadline, say we need 2 more days.” 2. HIM resolves “Rahim” → {email: rahim@gmail.com, tag: colleague, history: ...}

Field	Detail
	3. Agent builds context-aware prompt with injected profile. 4. LLM generates draft with appropriate subject and body. 5. System presents draft: “Here’s the draft. Send? (yes/edit/cancel)” 6. User: “Yes.” 7. Handler API calls Gmail API <code>messages.send()</code> . 8. System confirms: “Sent to rahim@gmail.com.” 9. Action logged in InteractionLog for Rahim.
Alternate A	Flow 6a. User: “Edit — change subject to ‘Deadline Extension Request’.” → System updates draft → re-presents → loops to step 5.
Alternate B	Flow 7a. Gmail API error → System displays error message, draft preserved for retry.
Alternate C	Flow 2a. “Rahim” not found in contact DB → System asks: “I don’t have an email for Rahim. What is it?”
Postcondition	Email delivered; InteractionLog entry created for Rahim.

5.3.3 UC-08: Create Event + Meet Link

Field	Detail
Use Case ID	UC-08
Name	Create Event with Meet Link
Actor(s)	Primary User, Google Calendar API
Precondition	Google Calendar OAuth valid; HIM populated
Main Flow	Success 1. User: “Schedule a project review with Rahim and Karim on Friday at 3 PM for 1 hour.” 2. HIM resolves “Rahim” and “Karim” to their stored emails. 3. System checks Friday 3 PM for conflicts. 4. Handler API creates event with <code>conferenceData.createRequest</code> . 5. Attendees added; Calendar sends invite emails.

Field		Detail
		6. System responds: “Meeting created. Meet: https://meet.google.com/xyz . Invites sent to rahim@ and karim@.”
		7. Action logged in HIM for both contacts.
Alternate A	Flow	3a. Conflict found at 3 PM → “You have ‘CSE Lab’ at 3 PM on Friday. Create anyway? (yes/pick another time)”
Alternate B	Flow	2a. Contact not found → System asks for email.
Postcondition		Event in Google Calendar; invites sent; Meet link accessible.

5.3.4 UC-13: Scheduled News Digest

Field		Detail
Use Case ID		UC-13
Name		Scheduled News Digest
Actor(s)		Scheduler, Telegram API, News Sources
Precondition		At least one NewsSource configured; Telegram chat ID set
Main Flow	Success	1. Cron fires at configured time. 2. Scraper fetches all active NewsSource URLs. 3. New articles extracted; deduplicated against delivered set. 4. Keyword filter applied. 5. LLM generates digest with article summaries. 6. Digest formatted in Telegram MarkdownV2 with in-line links. 7. Handler API sends message via Telegram Bot API. 8. Delivered article URLs stored in deduplication log.
Alternate A	Flow	2a. Scraper blocked on a source → Skip that source; log error; continue with others.
Alternate B	Flow	7a. Telegram API error → Retry once after 60 seconds; if still failed, log and skip.

Field	Detail
Postcondition	Digest delivered to Telegram; deduplication log updated.

Chapter 6

Swimlane Diagrams

6.1 Overview

Swimlane diagrams show process flow partitioned by responsible component. The following diagrams cover the five most important cross-cutting scenarios in the system.

6.2 Swimlane 1: Full Email Send with Context Injection

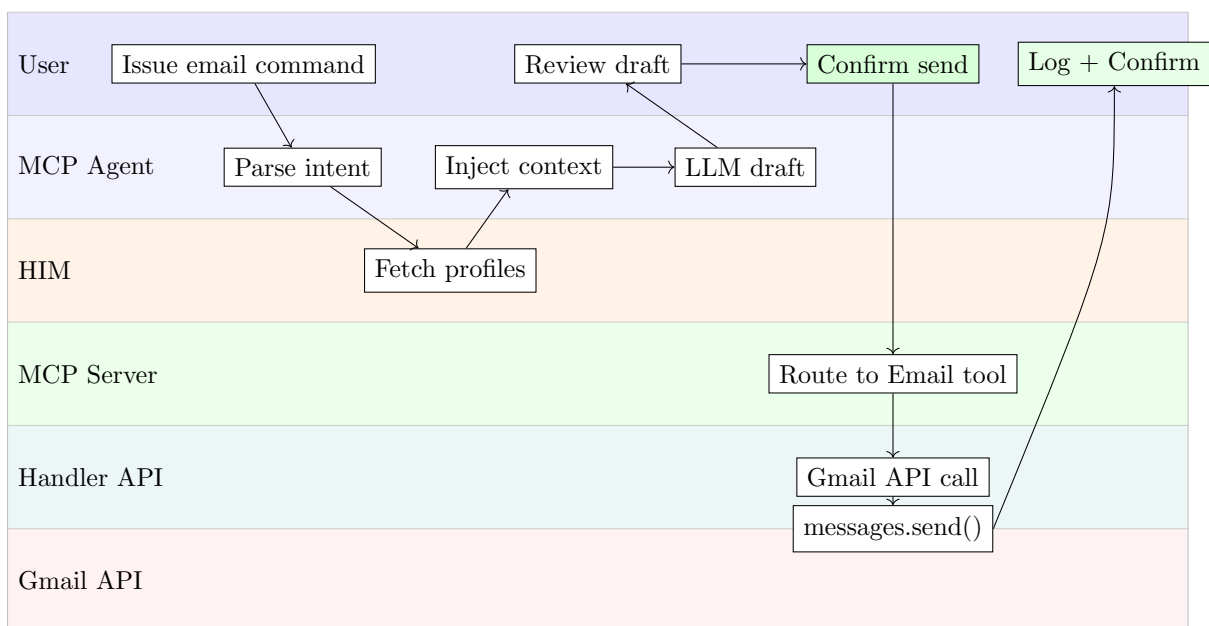


Figure 6.1: Swimlane — Email System with MCP + HIM Context Injection

6.3 Swimlane 2: Create Meeting + Meet + Invite Contacts

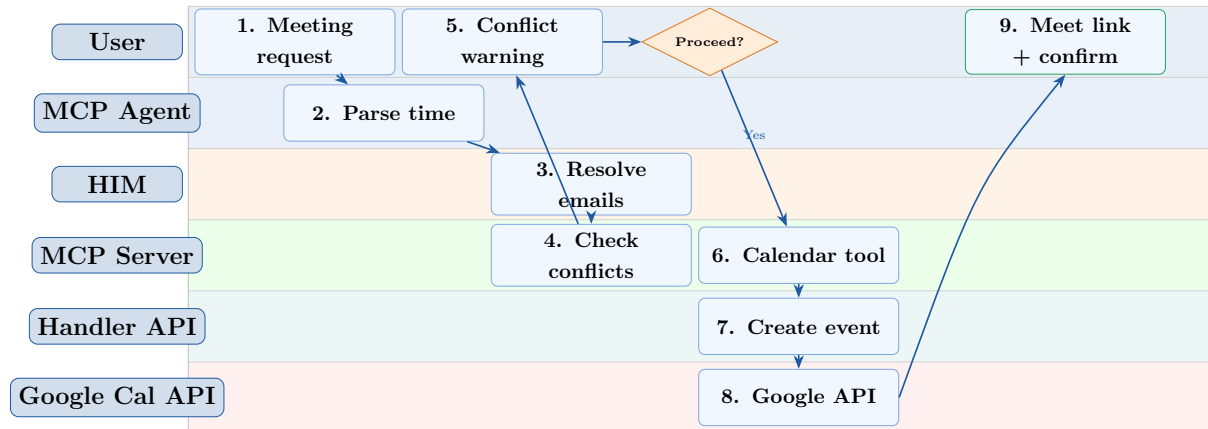


Figure 6.2: Swimlane 2 — Meeting Creation with Conflict Check (Improved Spacing)

6.4 Swimlane 3: Automated News Digest to Telegram

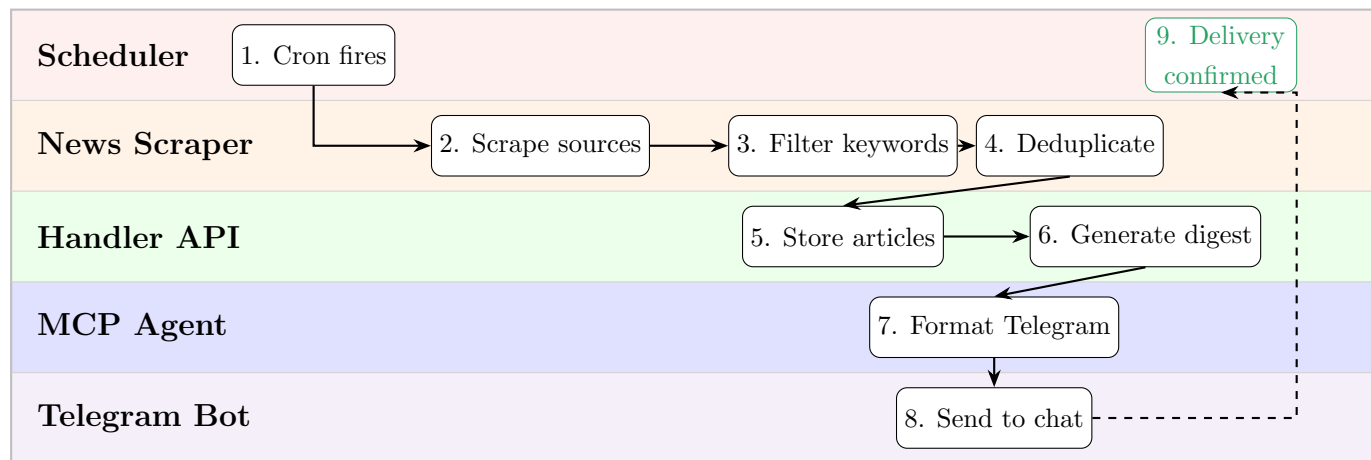


Figure 6.3: Automated Scheduled News Digest

6.5 Swimlane 4: Telegram Command Handling (/summary)

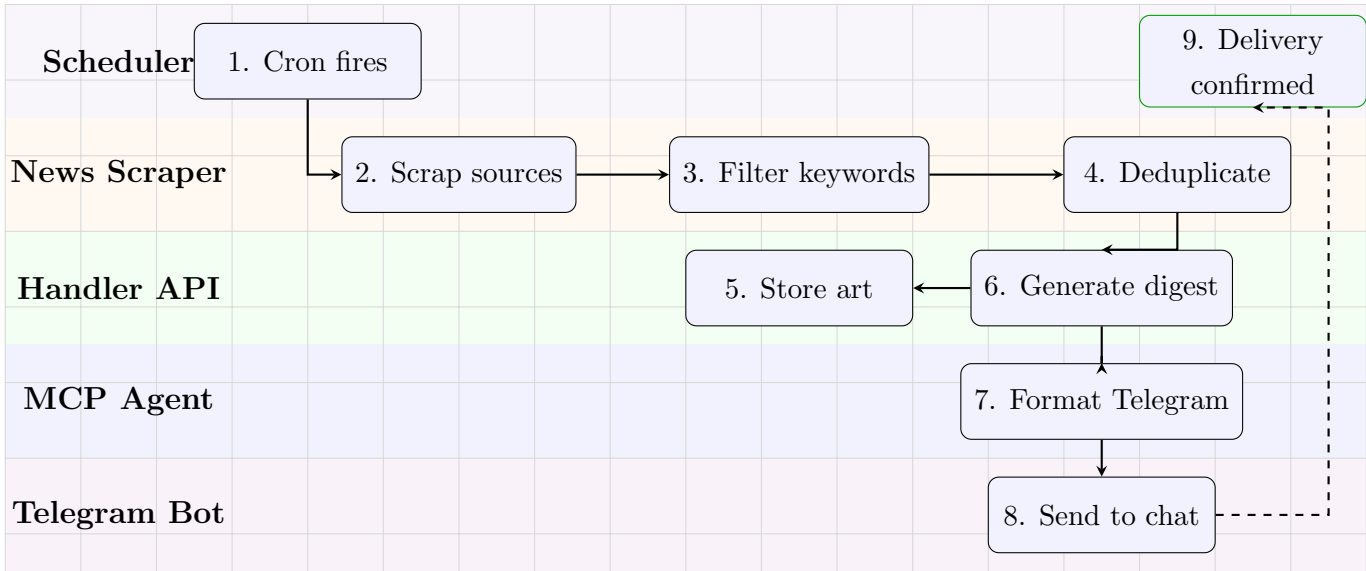


Figure 6.4: Swimlane 3: Automated News Digest to Telegram (Fixed Spacing)

Chapter 7

Structural Modelling

7.1 Layered Architecture Diagram

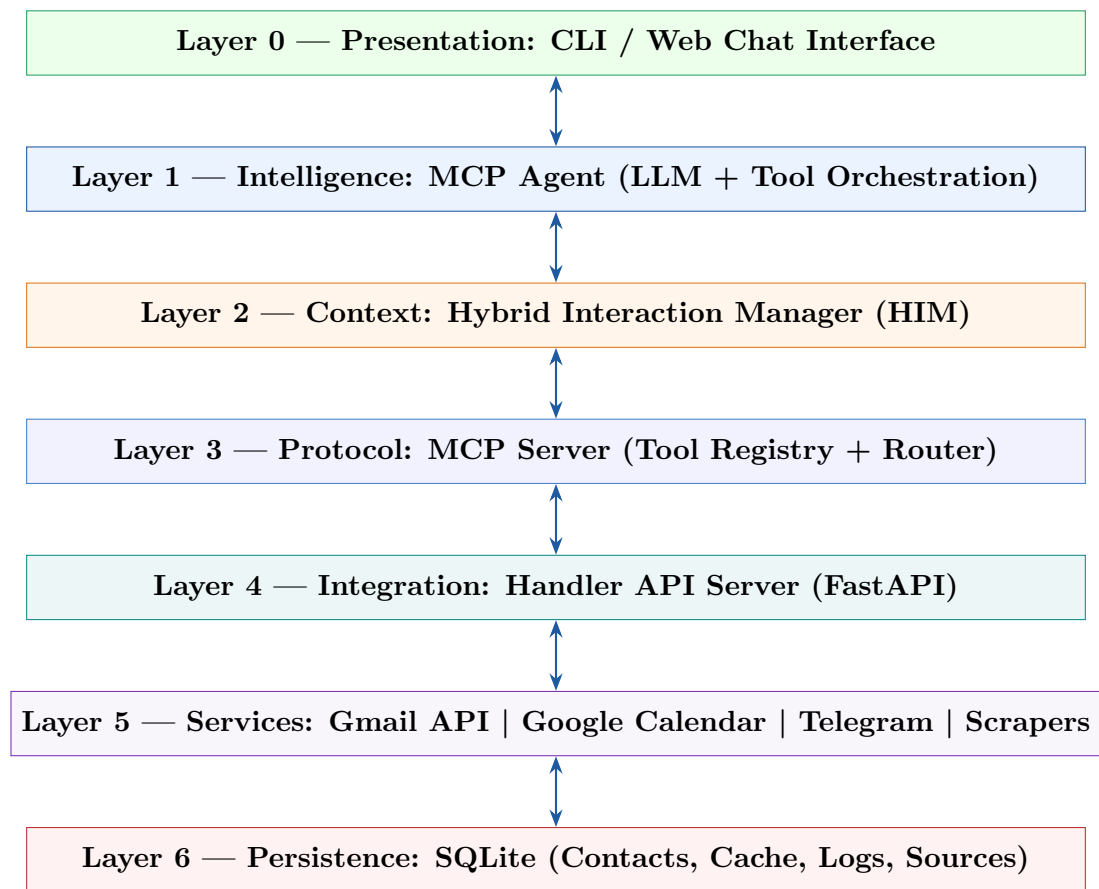


Figure 7.1: Layered Architecture — Seven-Layer Stack

7.2 Component Diagram

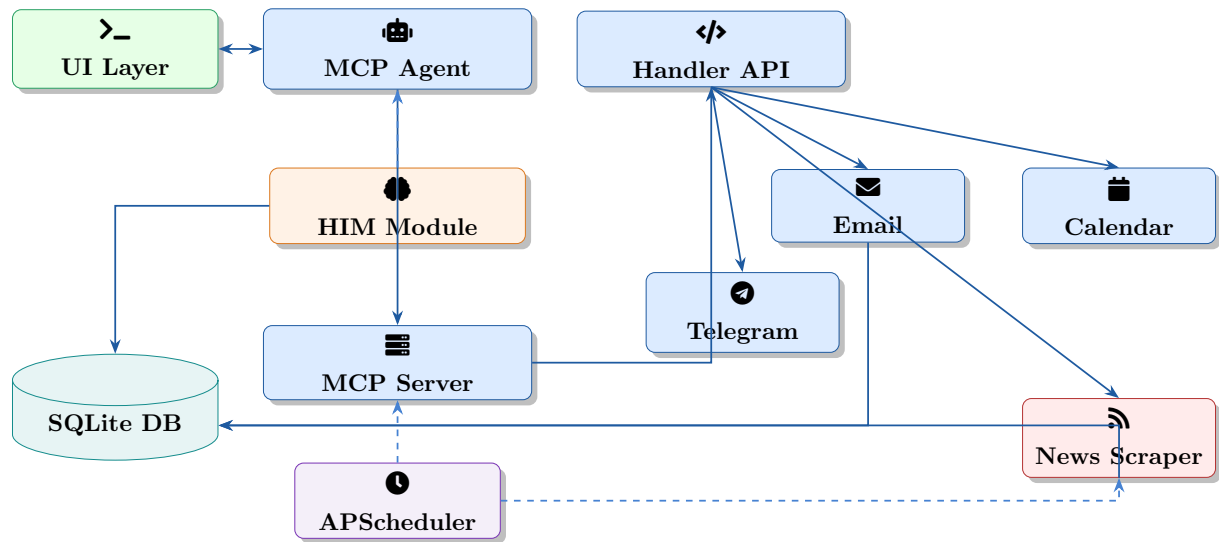


Figure 7.2: Dynamic Component Diagram (Connected Nodes)

7.3 Entity Relationship Diagram

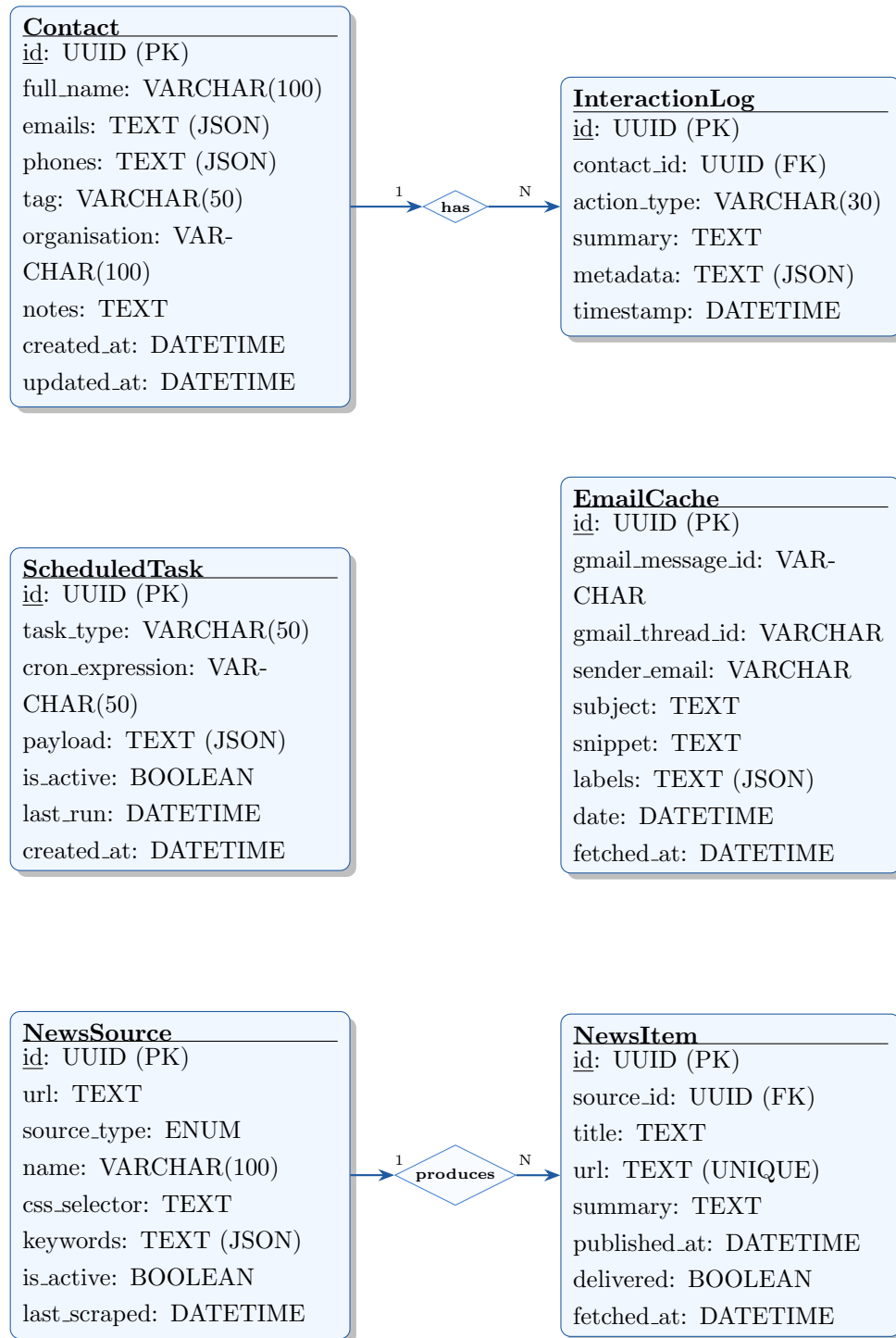


Figure 7.3: Entity Relationship Diagram — Persistence Layer

Chapter 8

Class Design

8.1 Full Class Diagram

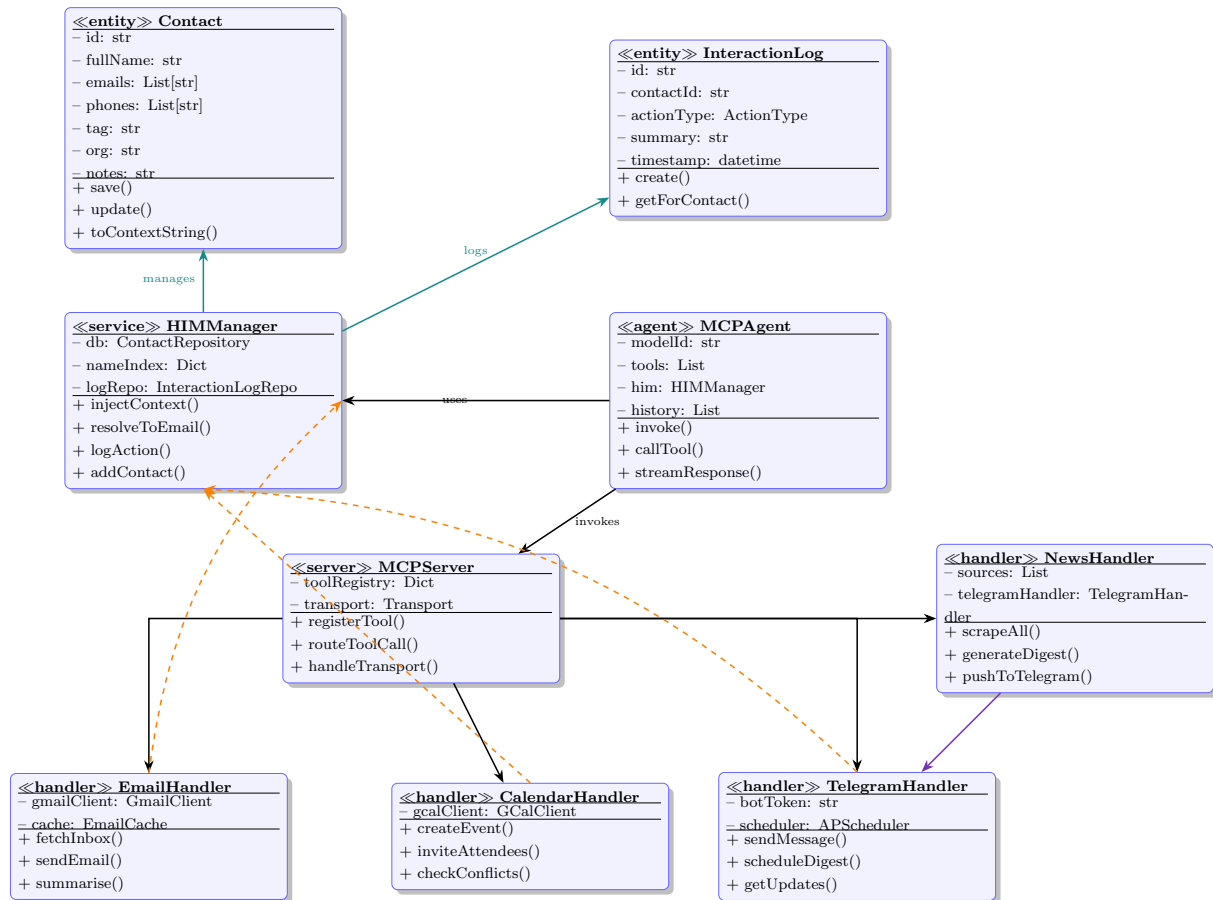


Figure 8.1: Class Diagram — Core Components and Dependencies

Chapter 9

Data Flow Diagrams

9.1 Level-0 DFD — Context Diagram

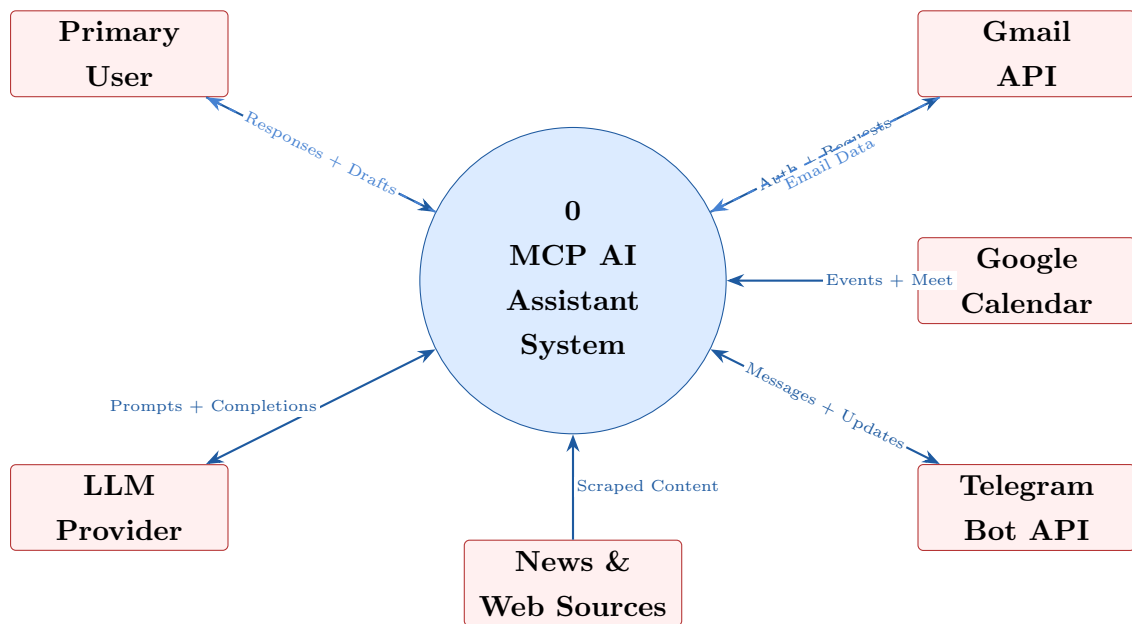


Figure 9.1: Level-0 DFD — System Context Diagram

9.2 Level-1 DFD — Process Decomposition

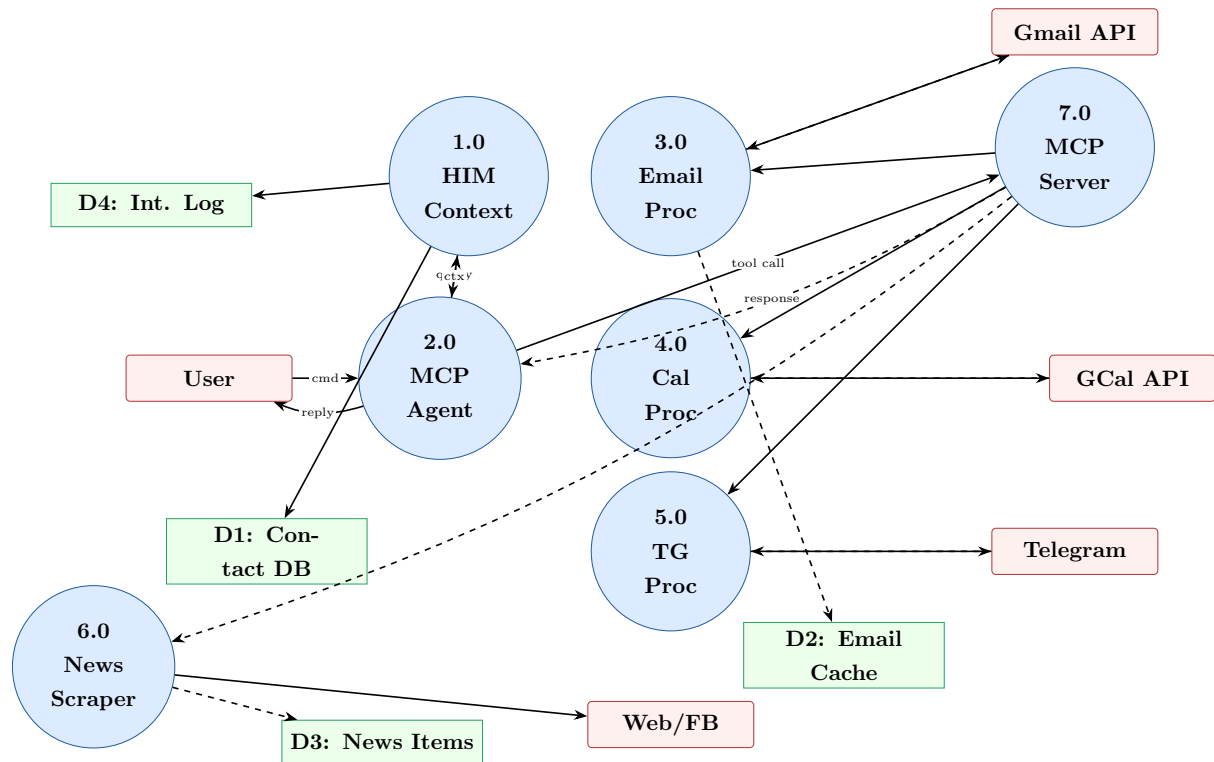


Figure 9.2: Level-1 DFD — Internal Process Decomposition

9.3 Level-2 DFD — Email Process (P3) Expansion

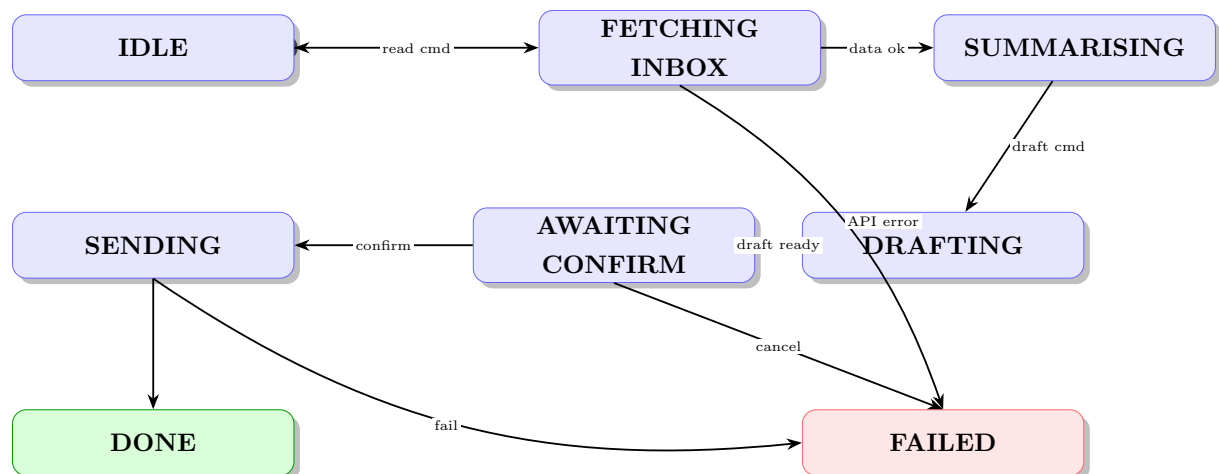


Figure 9.3: State Machine: Email Lifecycle

Chapter 10

Behavioral Modelling

10.1 State Machine 1: Email Lifecycle

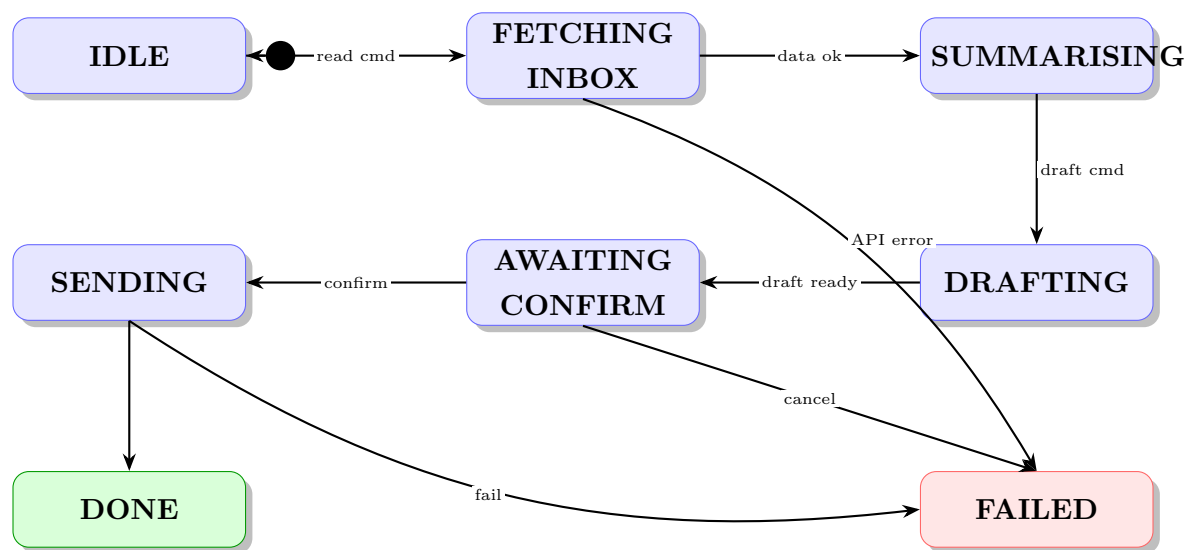


Figure 10.1: State Machine: Email Lifecycle

10.2 State Machine 2: HIM Context Injection Lifecycle

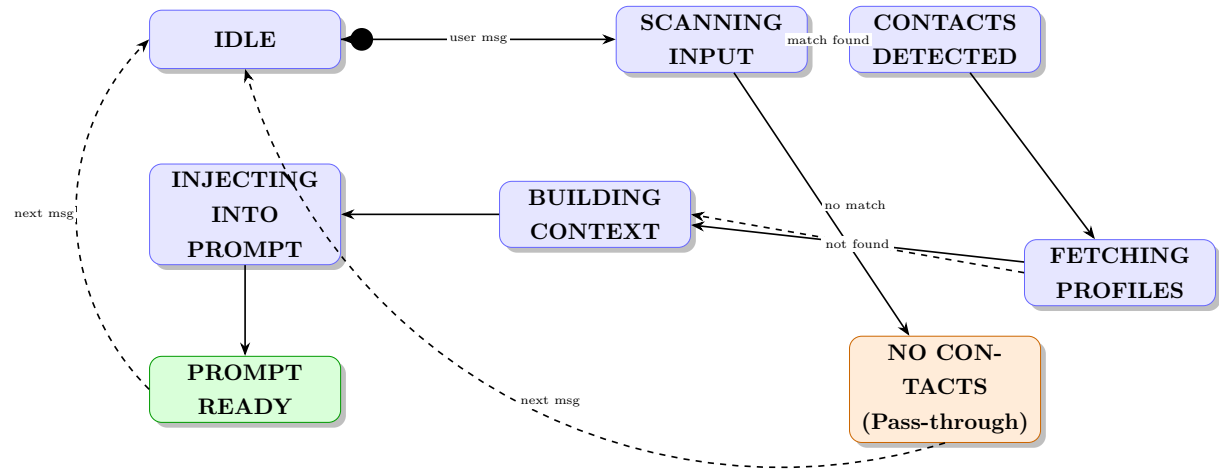


Figure 10.2: State Machine: HIM Context Injection Lifecycle

10.3 Sequence Diagram 1: Full Email Send Flow

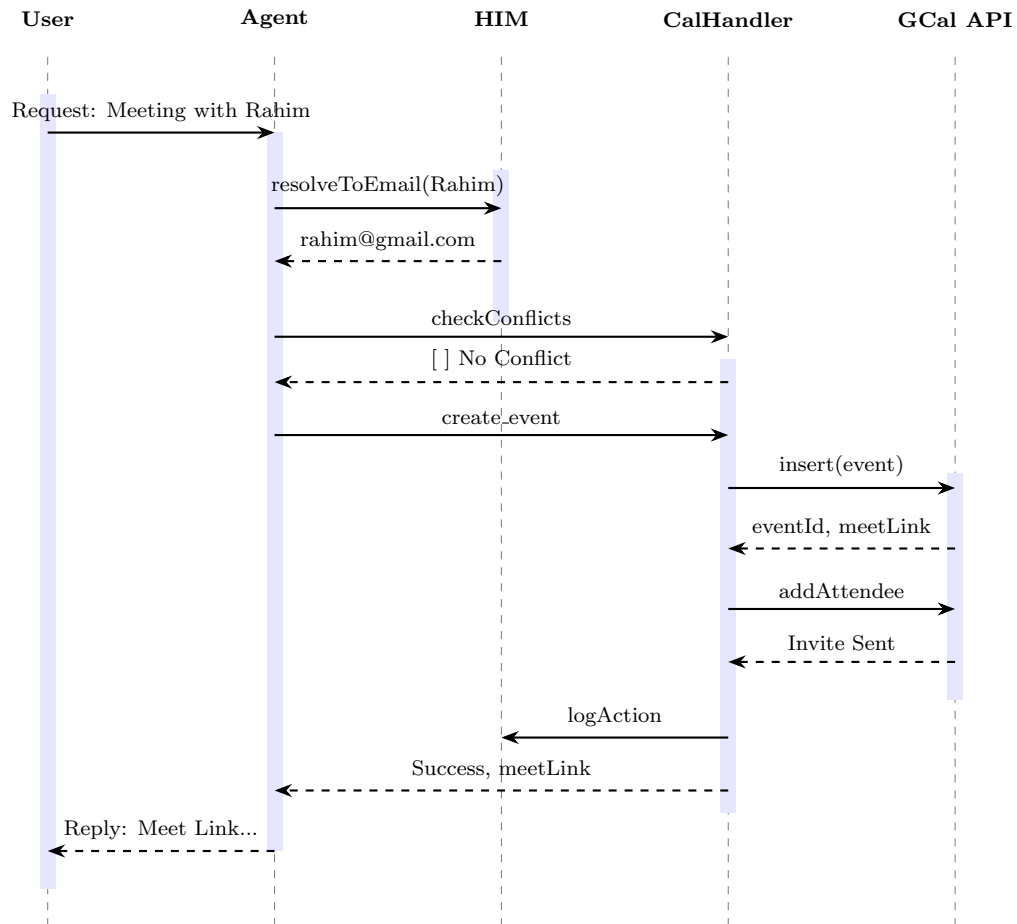


Figure 10.3: Sequence Diagram 1 — Calendar Event, Meet Link, and Invite

10.4 Sequence Diagram 2: Calendar Event + Meet + Invite

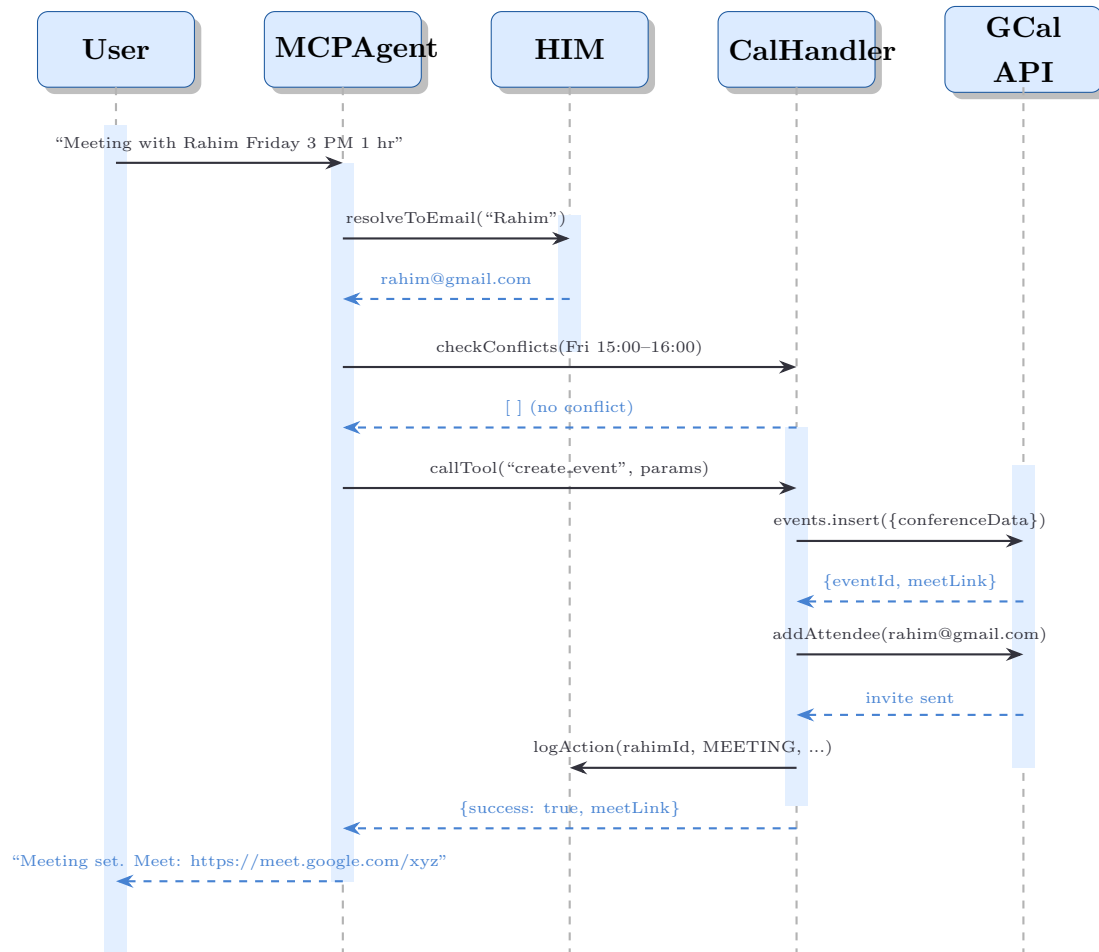


Figure 10.4: Sequence Diagram 2 — Calendar Event, Meet Link, and Invite

10.5 Sequence Diagram 3: Telegram /summary Command

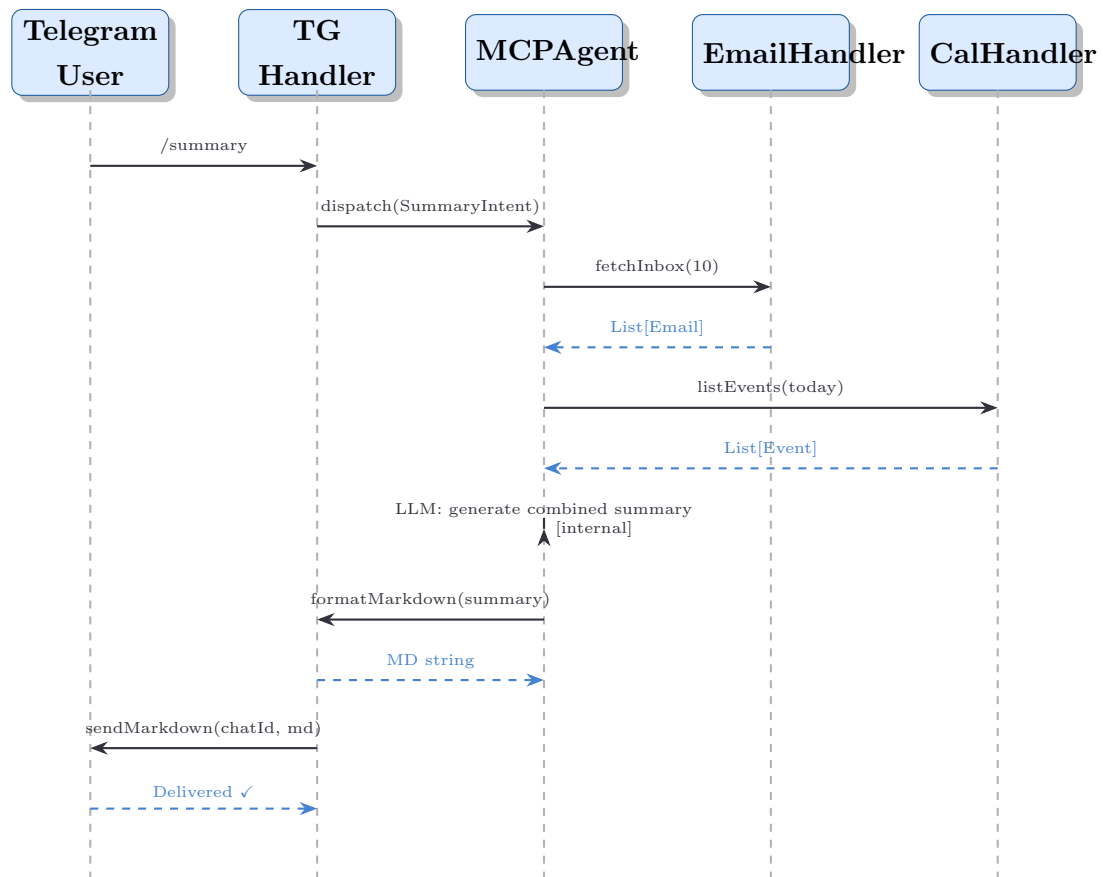


Figure 10.5: Sequence Diagram 3 — Telegram /summary Command Flow

10.6 Activity Diagram: HIM Context Injection Decision Flow

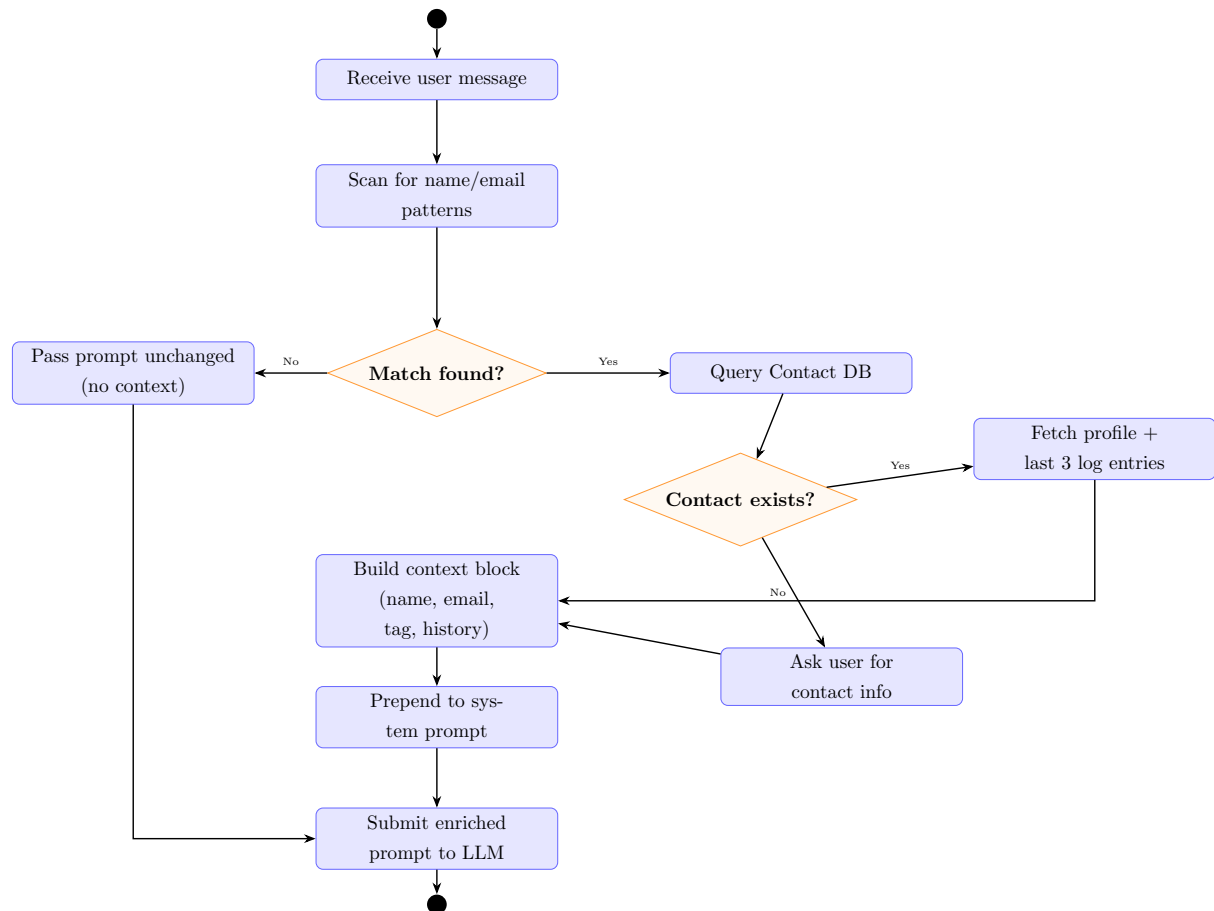


Figure 10.6: Activity Diagram — HIM Context Injection Decision Flow

Chapter 11

Non-Functional Requirements

11.1 Overview

Non-functional requirements (NFRs) define the quality attributes of the system. They constrain how the system behaves rather than what it does. All NFRs are testable and measurable where possible.

11.2 Performance Requirements

ID	Metric	Requirement
NFR-P1	Response Latency — Simple	For conversational turns requiring no external API call (e.g., contact lookup, history query), the end-to-end response time shall not exceed 3 seconds under normal load conditions.
NFR-P2	Response Latency — API	For turns requiring a single external API call (fetch email, create event), end-to-end latency shall not exceed 8 seconds .
NFR-P3	Response Latency — Multi-step	For complex multi-step operations (create event + invite + email confirmation), latency shall not exceed 15 seconds .
NFR-P4	Email Fetch Throughput	Fetching the 20 most recent inbox emails including metadata shall complete within 5 seconds under standard network conditions.
NFR-P5	Scheduled Task Precision	Cron-triggered tasks (digest, news push) shall fire within ± 60 seconds of their configured cron time.
NFR-P6	News Scraper Throughput	The scraper shall process all active sources (up to 20) within 30 seconds per cycle, using async HTTP.
NFR-P7	Context Injection Overhead	HIM context injection (name detection + DB query + prompt building) shall add no more than 200 ms to the total response latency.

ID	Metric	Requirement
NFR-P8	Concurrent Tool Calls	The MCP Server shall support parallel tool-call execution for independent tools (e.g., fetch inbox + fetch calendar simultaneously).

11.3 Security Requirements

ID	Category	Requirement
NFR-S1	Token Encryption	All OAuth 2.0 access and refresh tokens shall be stored encrypted at rest using AES-256 via Python's <code>cryptography</code> library. Tokens shall never appear in log files, console output, or version control.
NFR-S2	Secret Management	All secrets (LLM API key, Telegram Bot token, Google client secret) shall be loaded exclusively from environment variables or a <code>.env</code> file excluded from version control via <code>.gitignore</code> .
NFR-S3	OAuth Scope Minimisation	Gmail OAuth scope shall request only <code>gmail.modify</code> (not <code>mail.google.com</code>). Calendar scope shall request only <code>calendar.events</code> . Principle of least privilege applies.
NFR-S4	Single-User Isolation	The system shall enforce single-user authentication. All data (contacts, logs, cache) is owned by the single authenticated user. No shared data paths exist.
NFR-S5	Confirmation Before Write	All state-mutating operations (send email, create event, delete contact, broadcast Telegram message) shall require a positive confirmation response from the user before execution.
NFR-S6	Input Sanitisation	All natural language input that flows into SQL queries shall pass through parameterised query interfaces (SQLAlchemy ORM). No raw string concatenation in SQL.
NFR-S7	HTTPS Only	All HTTP communication with external APIs (Gmail, GCal, Telegram, LLM provider) shall be over TLS 1.2 or higher.

ID	Category	Requirement
NFR-S8	Audit Log	Every tool call (name, arguments, result, timestamp) shall be written to a structured audit log file (JSON lines format) on disk.

11.4 Reliability and Availability Requirements

ID	Category	Requirement
NFR-R1	Uptime	The Handler API server shall maintain 99% uptime during active use hours (06:00–24:00 local time), excluding scheduled maintenance windows of no longer than 30 minutes per week.
NFR-R2	Graceful Degradation	If any single external API (Gmail, Google Calendar, Telegram, news sources) becomes unavailable, the system shall: (a) log the error, (b) inform the user with a plain-language message, (c) continue operating all other modules unaffected.
NFR-R3	Data Persistence	All contact records, scheduled tasks, and news source configurations shall survive application restarts with zero data loss. SQLite WAL mode shall be enabled.
NFR-R4	Email Cache Resilience	If the email cache is corrupted or unreadable on startup, the system shall rebuild it from a fresh Gmail API fetch rather than crashing.
NFR-R5	Retry Policy	All external API calls shall implement exponential back-off with jitter: up to 3 retries, base delay 1 second, max delay 30 seconds.
NFR-R6	LLM Fallback	If the primary LLM provider (Anthropic Claude) returns a 5xx error, the system shall automatically retry once, then fall back to the secondary provider (OpenAI GPT-4o) if configured.
NFR-R7	Scheduled Task Recovery	If the scheduler process crashes and restarts, all scheduled tasks shall be reloaded from the Scheduled-Task table and re-registered without duplication.

11.5 Usability Requirements

ID	Category	Requirement
NFR-U1	Natural Language Interface	Users shall be able to accomplish all primary tasks using free-form natural language. No command memorisation shall be required for standard workflows.
NFR-U2	Error Clarity	All error messages surfaced to the user shall: (a) state what failed in plain English, (b) suggest a remediation step (e.g., “Try re-authenticating”), (c) not expose stack traces or internal identifiers.
NFR-U3	Onboarding	A new user shall be able to complete initial setup (OAuth auth, Telegram bot link, first contact add) within 10 minutes following the provided setup guide.
NFR-U4	Response Format	All agent responses shall be formatted for readability: bullet points for lists, bold for key information, plain prose for explanations. Telegram responses shall use MarkdownV2.
NFR-U5	Undo / Cancel	At any confirmation prompt, the user shall be able to type “cancel”, “no”, or “stop” to abort the pending action without side effects.
NFR-U6	Help Command	The system shall respond to “/help” or “What can you do?” with a structured list of all supported capabilities grouped by module.

11.6 Maintainability and Extensibility Requirements

ID	Category	Requirement
NFR-M1	Modular Architecture	Each integration module (Email, Telegram, Calendar, News, HIM) shall be independently startable, testable, and deployable without modifying other modules. Each module shall expose its capabilities exclusively via the Handler API.
NFR-M2	Tool Schema Versioning	All MCP tool definitions shall include a version field. Breaking changes to a tool’s input/output schema shall increment the version. The prior version shall remain active for one release cycle.

ID	Category	Requirement
NFR-M3	Structured Logging	All application events (startup, tool calls, API requests, errors, scheduled triggers) shall be logged in JSON Lines format with fields: timestamp, level, module, event, duration_ms, error (if any).
NFR-M4	Test Coverage	Unit test coverage for all Handler API endpoints shall meet or exceed 80%. Integration tests shall cover all primary use case flows (UC-01 through UC-13).
NFR-M5	Configuration-Driven	All configurable parameters (news scrape interval, digest schedule, email fetch count N, LLM model ID, context injection depth) shall be readable from a <code>config.yaml</code> or environment variables without code changes.
NFR-M6	Dependency Isolation	Each module shall declare its external dependencies in a <code>requirements.txt</code> section. A new developer shall be able to install and run any single module in isolation within 5 minutes.
NFR-M7	API Documentation	FastAPI's automatic OpenAPI/Swagger documentation shall be enabled and accessible at <code>/docs</code> . All endpoints shall include summary, description, and example request/response schemas.

11.7 NFR Summary Radar



Figure 11.1: NFR Quality Attribute Radar — Target Profile (1=Low, 5=High)

Chapter 12

Constraints & Assumptions

12.1 Technical Constraints

Technical Constraints

1. **Single-User Architecture (v1.0):** The entire system is designed for exactly one authenticated user. All OAuth tokens, contacts, email caches, and preferences belong to a single identity. Multi-tenancy introduces significant complexity (token isolation, data partitioning, per-user rate limiting) and is explicitly deferred to v2.0.
2. **Google API Quotas:** The Gmail API enforces a 250 quota units per user per second limit, with a daily cap of 1 billion units. Bulk operations (fetching 50 emails) must use batch requests and respect back-off headers. The system must cache aggressively to avoid re-fetching unchanged data.
3. **Telegram Bot Cannot Initiate Conversations:** The Telegram Bot API does not permit a bot to send the first message to a user who has not previously interacted with the bot. The user must send at least one message or `/start` command before the bot can reach them. This constrains initial setup; the user must initiate contact with the bot.
4. **Facebook Page Scraping Fragility:** Facebook actively resists scraping through JavaScript rendering, login walls, and frequently changing class names. The scraper can only reliably access public page data under optimal conditions. The CSS selectors used are subject to breaking on any Facebook UI update. RSS fallback is mandatory.
5. **LLM Context Window Upper Bound:** Even with a 200k-token context window (Claude 3.5 Sonnet), injecting full email threads, full contact histories, and full calendar data simultaneously is infeasible. The HIM must enforce a strict context budget: maximum 3 contact profiles per turn, maximum 5 recent emails in cache, maximum 7 upcoming events.
6. **No Real-Time Gmail Push (v1.0):** Implementing Gmail Push Notifications requires a publicly accessible HTTPS webhook endpoint registered with Google

Cloud Pub/Sub. This adds deployment complexity. v1.0 uses polling (fetch on demand or on schedule). Push notifications are a v2.0 enhancement.

7. **Python Runtime Requirement:** The Handler API and MCP Server require Python 3.11 or higher for the `asyncio` task groups and match-statement features used. Downgrading to Python 3.9/3.10 would require code changes.
8. **Internet Dependency:** All core functions (email, calendar, LLM, Telegram) require a live internet connection. There is no meaningful offline mode. If connectivity is lost, the system shall queue outbound actions and retry on reconnect for Telegram; all other operations shall fail gracefully with a user notification.

12.2 Business and Regulatory Constraints

Constraint		Detail
Google	API	All Gmail and Google Calendar API usage must comply with Google's API Terms of Service. Automated bulk email sending that could be construed as spam is prohibited. The system shall not be used for mass marketing.
Terms of Service		
Telegram	Bot	Telegram bots must comply with the Telegram Bot Terms of Service. Bots may not be used for spamming, harassment, or sending unsolicited messages. The system shall only send messages to chats where the user has explicitly authorised the bot.
Policy		
Data Privacy		All contact data stored by the HIM is personal data under applicable privacy laws (GDPR if the user is in the EU, PDPO if applicable). The user is the sole data controller. No contact data shall be transmitted to third parties other than the configured external APIs (Google, Telegram) for the purpose of the user's own actions.
LLM	Provider	Prompts sent to the LLM provider (Anthropic, OpenAI) must comply with their usage policies. Email contents and contact information transmitted in prompts constitute personal data; the user accepts this by configuring the system.
Terms		

Constraint	Detail
Academic Integrity	In- This project is an academic submission for CSE-3104. All code, diagrams, and documentation are original work by Team Axis. Use of third-party libraries is permitted and attributed in the project README.

12.3 Assumptions

ID	Assumption
A-01	The user has an active Gmail account with full access rights and has enabled the Gmail API in Google Cloud Console with OAuth 2.0 credentials (client ID + client secret) available.
A-02	The user has a Google Calendar account linked to the same Google account as Gmail. Calendar events created by the system are visible in the user's primary calendar.
A-03	A Telegram Bot has been created via BotFather and the bot token is available at deployment time. The user's personal Telegram account chat ID is known and configured.
A-04	The LLM provider API key (Anthropic or OpenAI) is active, funded, and has rate limits sufficient for the user's expected usage volume (estimated 100–500 API calls per day).
A-05	The deployment server (local machine or VPS) has stable internet connectivity, no firewall rules blocking outbound HTTPS to Google, Telegram, Anthropic/OpenAI, or news domains.
A-06	Target news websites and Facebook pages configured by the user are publicly accessible without login or JavaScript rendering.
A-07	The user has basic familiarity with running a terminal, setting environment variables, and navigating a CLI. No GUI is required for operation.
A-08	The user operates in a single timezone. All datetime handling in the system uses the user's local timezone as configured in <code>config.yaml</code> .
A-09	Email composition language is English. The LLM's default language for drafted emails is English unless the user explicitly specifies otherwise in their command.

ID	Assumption
A-10	Google Meet link generation is available on the user's Google account tier. Meet links require a Google Workspace or personal Google account with Meet access.
A-11	The SQLite database file is stored on a local disk with sufficient space (at minimum 1 GB free). Data backup is the user's responsibility.
A-12	Facebook's public page scraping will be functional for at least the initial project demonstration. Long-term reliability of this feature is explicitly not guaranteed.

12.4 Dependencies

Dependency	Version	Role
Python	3.11+	Runtime
FastAPI	0.111+	Handler API framework
anthropic (SDK)	0.28+	MCP Agent LLM calls
google-api-python-client	2.x	Gmail + Calendar API client
google-auth-oauthlib	1.x	OAuth 2.0 flow
python-telegram-bot	21.x	Telegram Bot integration
SQLAlchemy	2.x	ORM for SQLite
APScheduler	3.x	Cron job scheduling
httpx	0.27+	Async HTTP for scrapers
BeautifulSoup4	4.12+	HTML parsing
pydantic	2.x	Data validation
cryptography	42.x	Token encryption at rest
pytest	8.x	Test framework

Chapter 13

Conclusion

13.1 Document Summary

This Software Requirements Specification has comprehensively defined the **AuraMCP** — a context-aware personal automation platform built on the Model Context Protocol. The document has covered:

- **Problem Statement (Ch. 1):** Quantified the fragmentation problem, mapped root causes, and defined the solution boundary.
- **Inception (Ch. 2):** Established project vision, technology stack, feasibility across three dimensions, stakeholder analysis, full risk register, and a week-by-week milestone plan.
- **Elicitation (Ch. 3):** Documented 34 functional requirements across five modules using MoSCoW prioritisation, with full requirement descriptions meeting IEEE 830 standards.
- **Use Case Modeling (Ch. 4):** Defined 13 use cases with a master diagram and full tabular descriptions for the three most complex scenarios.
- **Swimlane Diagrams (Ch. 5):** Four process-flow diagrams covering email send, meeting creation, news digest, and Telegram command handling.
- **Structural Modelling (Ch. 6):** Seven-layer architecture, detailed component diagram, and a six-entity ER diagram.
- **Class Design (Ch. 7):** Full class diagram with eight domain classes, all attributes, all methods, and inter-class relationships.
- **Data Flow Diagrams (Ch. 8):** Level-0, Level-1, and Level-2 DFDs covering all process decompositions.
- **Behavioral Modelling (Ch. 9):** Two state machines, three sequence diagrams, and an activity diagram.
- **Non-Functional Requirements (Ch. 10):** 33 NFRs across performance, security,

reliability, usability, and maintainability, with a quality radar chart.

- **Constraints & Assumptions (Ch. 11):** Eight technical constraints, five regulatory constraints, 12 assumptions, and a full dependency table.

13.2 Key Design Principles

Core Design Philosophy — Five Pillars

1. **♥ Context First:** Every agent action is preceded by HIM context injection. The assistant always knows *who* it is dealing with before it reasons or acts. This transforms it from a stateless chatbot into a true personal assistant with memory.
2. **► Confirmation Before Action:** No email is sent, no calendar event is created, no contact is deleted, and no Telegram message is broadcast without explicit user confirmation. This is non-negotiable and enforced at the Handler API layer, not just the agent layer.
3. **⚡ Protocol-Native Extensibility:** Every capability is a first-class MCP tool with a versioned JSON schema. Adding WhatsApp, Notion, or Slack is a matter of adding new Handler API endpoints and registering new tools with the MCP Server — the agent and HIM require no changes.
4. **⚠ Graceful Degradation:** Module-level failures are isolated. A Gmail outage does not disable Calendar or Telegram. A Facebook scraping block does not halt news from other sources. Users are informed clearly and the rest of the system continues operating.
5. **🗄 Persistent Intelligence:** Context, contacts, interaction history, scheduled tasks, and news sources persist across restarts. The system grows smarter with every interaction as the HIM accumulates richer profiles.

13.3 Expected Outcomes and Impact

Upon successful implementation, the AuraMCP will deliver the following measurable outcomes:

Outcome	Description
Time Savings	Estimated reduction of 1.5–2.5 hours per day in manual communication and scheduling overhead through automation of routine tasks.

Outcome	Description
Zero Missed Meetings	Automated conflict detection, contact-linked invites, and configurable reminders eliminate scheduling failures.
Context-Rich Communication	Every email and calendar invite benefits from HIM-injected relationship context, improving tone, accuracy, and completeness.
Proactive Information Delivery	Scheduled news digests and daily summaries ensure the user stays informed without active effort.
Extensible Foundation	The MCP-native architecture serves as a reference implementation for personal AI assistant platforms, demonstrating clean separation of concerns across intelligence, protocol, and integration tiers.
Academic Contribution	Demonstrates applied use of IEEE 830 SRS methodology, UML modelling, and full-stack system design in the context of a cutting-edge AI architecture (MCP) for CSE-3104 coursework evaluation.

13.4 Future Work Roadmap

Version	Timeline	Planned Additions
v1.1	Month 2	Gmail Push Notifications (real-time inbox updates via Pub/Sub); improved HTML email rendering; attachment metadata support
v2.0	Month 4	Multi-user tenancy with per-user OAuth token isolation; Web-based React dashboard UI; WhatsApp Business API integration
v2.1	Month 6	Voice input via Whisper API; voice output via TTS; recurring calendar events; smart scheduling with free/busy queries

Version	Timeline	Planned Additions
v3.0	Year 2	Proactive agent: initiates reminders and suggestions without user prompting based on learned behavioural patterns; on-device LLM option for privacy-sensitive users
